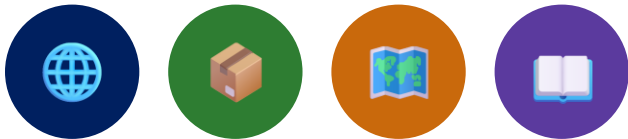


*"A well-designed API is a promise you keep
for years, not a guess you make once."*

MODULE 13

REST API Design with Java

Design resource-based, standards-correct REST APIs that enterprise clients can trust.





MODULE 13 OVERVIEW

Design and document a resource-based REST API for an enterprise HR system.

Scenario: model, name, and document the Employee Management API's resources and endpoints end to end.

DURATION & LAB



3 Hours Theory

REST constraints, resource design, HTTP semantics, status codes, and OpenAPI.



1 Hour Guided Practice

Design the Employee resource model, URIs, and an OpenAPI outline together.



Scenario

An Employee Management API serving HR self-service, payroll, and a future mobile app.

MODULE ARC



Resources and URIs

Model the domain and name every resource consistently.



HTTP Semantics

Match every operation to the right method and status code.



Document with OpenAPI

Capture the whole design in a versioned OpenAPI contract.

KEY TAKEAWAY A REST API is a resource model plus disciplined HTTP usage, documented so consumers can rely on it.

WHY REST MATTERS IN MODERN ENTERPRISE APPLICATIONS

REST is the default choice for enterprise APIs because it reuses the web's own infrastructure.



Interoperability

HTTP and JSON let Java, .NET, Python, and JavaScript clients share one contract without custom protocols.



Scalability

Stateless requests let load balancers route to any instance -- no server-side session affinity required.



Evolvability

Resources can gain new fields and endpoints without breaking existing clients when versioning is disciplined.



Ecosystem Fit

Browsers, mobile apps, and other services already speak HTTP -- REST reuses infrastructure everyone has.

KEY TAKEAWAY REST succeeds in the enterprise because it reuses HTTP's caching, security, and tooling instead of inventing new plumbing.



Why REST Matters

in Modern Enterprise Applications

REST (Representational State Transfer) is the most widely adopted architectural style for building web APIs. It enables systems to communicate over standard HTTP, ensuring interoperability, scalability, and flexibility across diverse platforms and technologies.

KEY BENEFITS



Interoperability

REST uses standard HTTP and JSON, enabling integration across platforms and languages.



Scalability

Stateless and cacheable communication allows systems to scale efficiently.



Simplicity

Uniform, resource-based APIs are easy to understand, implement, and maintain.



Performance

Leverages HTTP caching, compression, and lightweight communication.



Flexibility

Supports multiple clients (web, mobile, IoT, partners) with the same API.



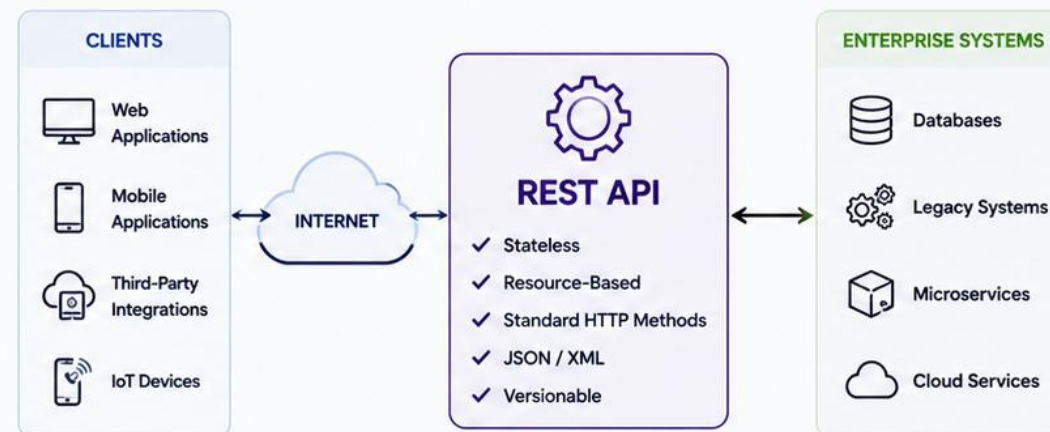
Evolvability

APIs can evolve with versioning while maintaining backward compatibility.



REST is the backbone of modern digital ecosystems, powering secure, scalable, and reliable communication between clients, services, and systems.

REST IN THE ENTERPRISE ARCHITECTURE



BUSINESS VALUE



Faster Time to Market



Better Partner Ecosystem



Lower Integration Costs



Stronger Security and Governance



Cloud-Ready and Future-Proof

MODULE LEARNING OBJECTIVES

By the end of this module, you will design and document a resource-based REST API from scratch.



Apply REST Constraints

Explain and apply client-server, stateless, cacheable, and uniform-interface constraints correctly.



Design Resources

Turn a business domain into well-named, hierarchical resource URIs.



Model Requests and Responses

Choose correct HTTP methods, status codes, and JSON error shapes for every operation.



Document with OpenAPI

Produce a versioned OpenAPI contract that other teams can build against.

KEY TAKEAWAY Every objective in this module points at one deliverable: a correctly designed Employee Management API contract.



Learning Objectives

By the end of this module, you will understand how to design well-structured, scalable, and enterprise-ready RESTful APIs using industry best practices and Java technologies.



In this module, you will be able to:

1



Understand REST principles and architectural constraints.

2



Design resource-based APIs with meaningful URIs.

3



Use appropriate HTTP methods and status codes.

4



Structure request and response payloads using JSON.

5



Implement pagination, filtering, and sorting in APIs.

6



Apply API versioning strategies effectively.

7



Document APIs using OpenAPI (Swagger).

8



Follow best practices for secure, scalable API design.

9



Identify common API design mistakes and how to avoid them.

10



Design enterprise-ready APIs that are maintainable and future-proof.



Goal: Design RESTful APIs that are consistent, secure, efficient, and aligned with enterprise standards.



BUSINESS SCENARIO: EMPLOYEE MANAGEMENT API

Every design decision in this module builds one API: employee records for an enterprise HR system.

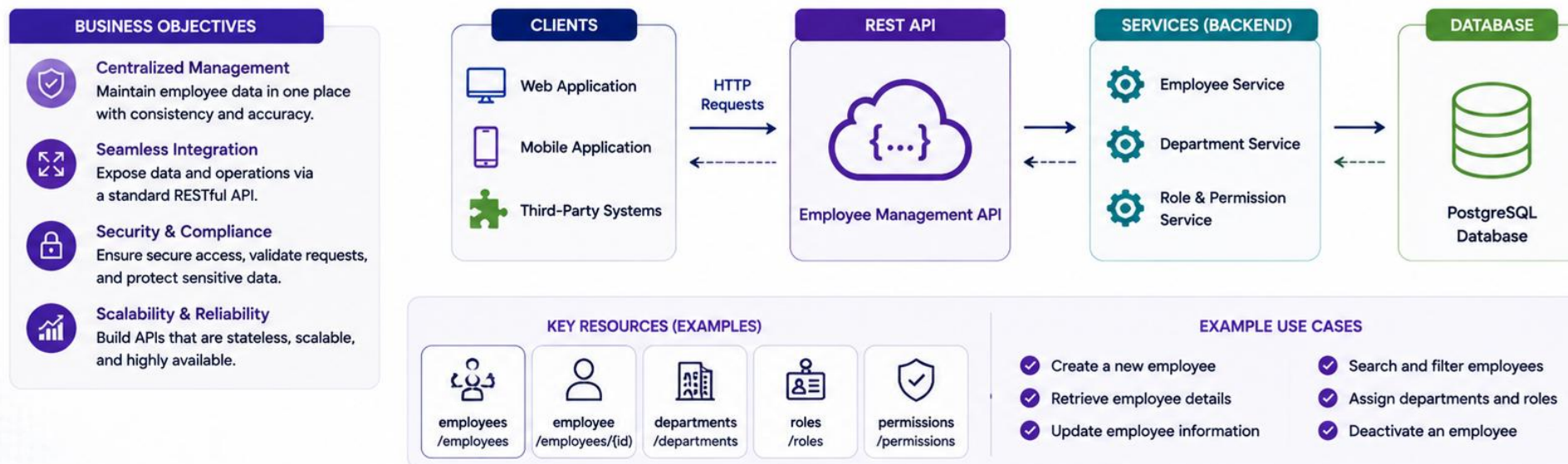
Resource	Example Endpoint	Notes
Employees	GET /api/v1/employees	Paginated, filterable, sortable list of employees
Single Employee	GET /api/v1/employees/{employeeId}	Retrieve one employee by identifier
Create Employee	POST /api/v1/employees	HR onboarding creates a new employee record
Replace Employee	PUT /api/v1/employees/{employeeId}	Full replacement of an employee record
Partial Update	PATCH /api/v1/employees/{employeeId}	Update salary or department only
Remove Employee	DELETE /api/v1/employees/{employeeId}	Offboard an employee (soft delete)

KEY TAKEAWAY Every endpoint in this module traces back to one resource: the Employee, exposed through /api/v1/employees.



Business Scenario: Employee Management API

Your organization needs a robust and secure API to manage employee information. Multiple applications (web, mobile, and third-party systems) will consume this API to perform employee related operations.



Outcome: By the end of this module, you will design a well-structured, secure, and scalable REST API to manage employees and support enterprise integrations.

WHAT IS AN API?

An API is the published contract that lets one piece of software use another's functionality.



A Defined Contract

The published set of operations a system exposes so other software can use it without seeing internals.



Producer and Consumer

The Employee API provides data and behavior; the HR portal or mobile app consumes it over the contract.



Many Styles, One Goal

REST, SOAP, gRPC, and GraphQL are all APIs -- they differ in transport and format, not in purpose.

KEY TAKEAWAY An API is a promise about behavior, not implementation -- consumers code against the contract, not the internals.



What is an API?

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other.

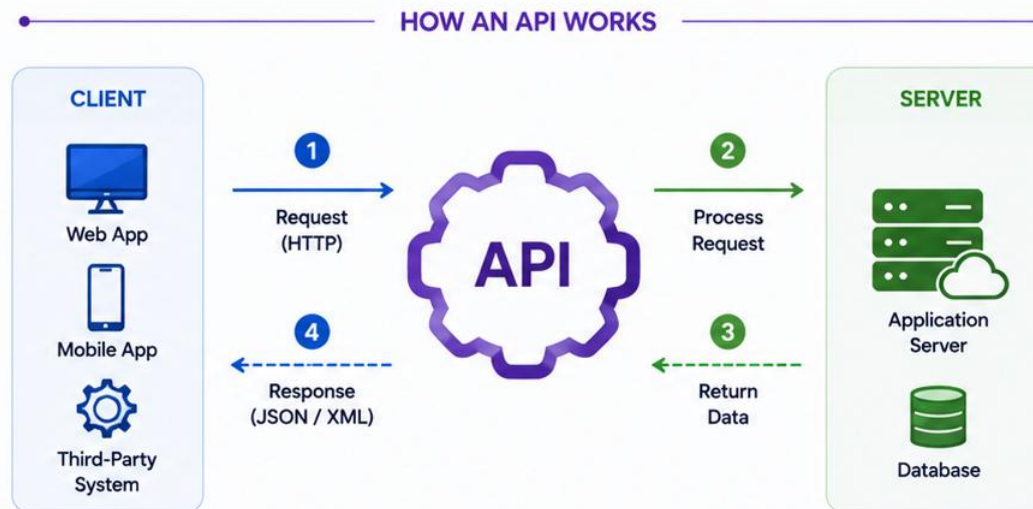


API acts as a contract

It defines how requests should be made, what data can be exchanged, and how responses are returned.

KEY POINTS

-  Enables communication between systems.
-  Promotes reusability and modularity.
-  Supports integration across platforms and devices.
-  Improves developer productivity.
-  Powers modern web, mobile, and enterprise applications.



REAL-WORLD EXAMPLES



Google Maps API
Integrate maps and location services into applications.



Weather API
Get real-time weather data for any location.



Payment Gateway API
Process payments securely in e-commerce applications.



Social Media API
Allow users to log in or share content through social platforms.



Travel API
Search flights, hotels, and travel information seamlessly.



In Short: An API is the **bridge** that connects applications, services, and data—enabling them to work together efficiently.

WHAT IS REST?

REST is an architectural style for exchanging resource representations over stateless HTTP requests.



Coined by Roy Fielding

REST (Representational State Transfer) comes from Fielding's 2000 dissertation on web architecture.



An Architectural Style

REST is a set of constraints, not a protocol or standard -- HTTP is simply the most common transport it runs over.



Resources as Representations

Clients exchange representations (JSON, XML) of server-side resources instead of manipulating the resources directly.

KEY TAKEAWAY REST is a style built on transferring resource representations over stateless, cacheable, uniform HTTP requests.



What is REST?

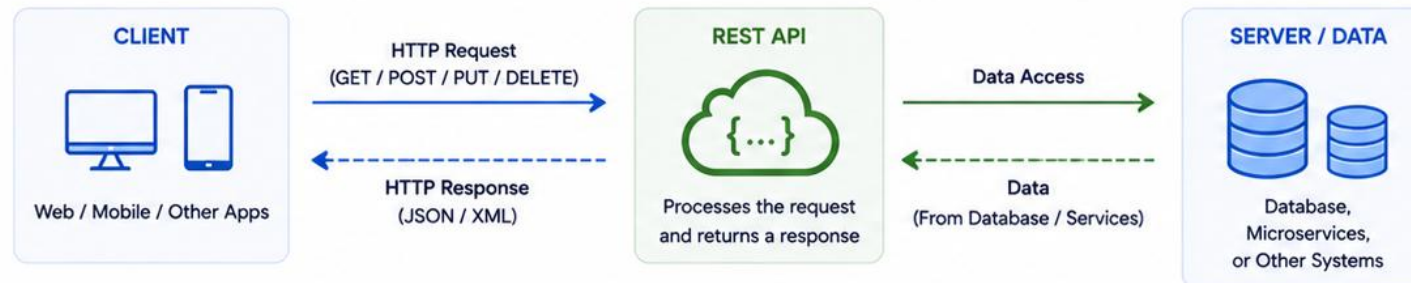
REST (Representational State Transfer) is an architectural style for designing networked applications. It uses standard HTTP methods to perform operations on resources, identified by URIs. REST is simple, stateless, scalable, and lightweight, making it the foundation of modern web APIs.

REST IN SIMPLE TERMS



REST is about using HTTP requests to access and manipulate resources on a server, just like how you click links and interact with web pages.

HOW REST WORKS



EXAMPLE

GET /api/employees/123

→ Returns details of employee with ID 123

POST /api/employees

→ Creates a new employee

REAL-WORLD EXAMPLE



E-commerce
APIs



Banking
APIs



Travel
APIs



Social Media
APIs



Cloud Service
APIs

KEY CHARACTERISTICS



Stateless

Each request from the client contains all information needed by the server.



Resource-Based

Everything is a resource, identified by a URI.



Standard HTTP Methods

Uses GET, POST, PUT, PATCH, DELETE for operations.



Representation

Resources are represented in formats like JSON or XML.



Scalable & Flexible

Easily scalable and can evolve over time without breaking clients.



REST is not a protocol or a library.

It is an **architectural style** that leverages HTTP and URI principles to build web services.

REST ARCHITECTURAL STYLE

REST reuses HTTP deliberately instead of layering a new protocol on top of it.



Style, Not Spec

There is no REST RFC to implement -- only constraints to satisfy, applied with varying strictness.



Built for the Web

REST deliberately reuses HTTP's verbs, status codes, and caching model instead of layering a new protocol on top.



Maturity Levels

Richardson's Maturity Model ranks APIs from plain RPC-over-HTTP up to full HATEOAS -- most enterprise APIs target Level 2.

KEY TAKEAWAY Treat REST as a checklist of constraints to satisfy deliberately, not a label to slap on any HTTP API.



REST Architectural Style

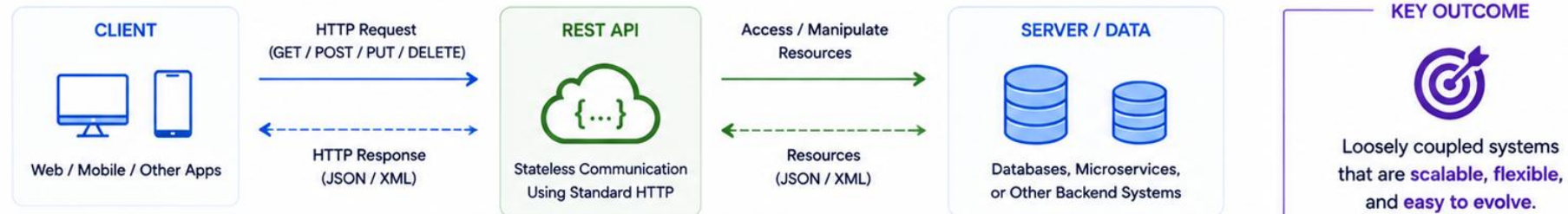
REST (Representational State Transfer) is an architectural style for designing networked applications. It defines a set of **constraints** that, when applied, lead to **scalable**, **maintainable**, and **reliable** web services.

WHAT MAKES IT AN ARCHITECTURAL STYLE?



REST is not a protocol or a library.
It is a **set of architectural constraints** applied to design web services that communicate over HTTP.

REST ARCHITECTURE OVERVIEW



REST IS GUIDED BY SIX ARCHITECTURAL CONSTRAINTS



In Essence: REST promotes **simplicity**, **scalability**, and **interoperability** by leveraging standard web technologies and a well-defined architectural style.

REST CONSTRAINTS OVERVIEW

Fielding defined six constraints; five are mandatory for a design to be considered RESTful.

Constraint	What It Requires
Client-Server	Separate UI concerns from data storage concerns
Stateless	Each request carries all context; the server holds no client session
Cacheable	Responses declare whether they can be stored and reused
Uniform Interface	Resources are identified and manipulated through a consistent set of verbs and representations
Layered System	Clients cannot tell whether they are talking to the origin server or an intermediary
Code on Demand (optional)	Servers may extend client behavior by transferring executable logic

KEY TAKEAWAY Five constraints are mandatory; code-on-demand is the only optional one -- most Java REST APIs never use it.

REST Constraints Overview

REST is not a protocol or standard. It is an architectural style defined by Roy Fielding. RESTful APIs follow a set of constraints that guide how systems should be designed.



WHY IT MATTERS

- Enables scalability and performance
- Improves simplicity and maintainability
- Ensures interoperability between heterogeneous systems
- Provides a consistent, predictable API experience

1		Client-Server	Separation of concerns between client (UI) and server (data/services) enables independent evolution and scalability.
2		Stateless	Each request from client to server must contain all the information needed. Server does not store any client context between requests.
3		Cacheable	Responses must define themselves as cacheable or non-cacheable to improve performance and scalability.
4		Uniform Interface	A consistent way of interacting with resources using standard HTTP methods, URIs, headers, and representations.
5		Layered System	The architecture can include multiple layers (e.g., load balancer, proxy, gateway) without the client needing to know.
6		Code on Demand (Optional)	Servers can extend client functionality by transferring executable code (e.g., scripts). This constraint is rarely used in modern APIs.



KEY TAKEAWAY: By following these constraints, RESTful APIs become scalable, reliable, and easy to evolve, supporting complex enterprise applications effectively.

CLIENT-SERVER ARCHITECTURE

The UI and the data/business logic evolve independently behind one stable contract.



Separation of Concerns

Angular owns presentation; Spring Boot owns data and business rules -- neither depends on the other's internals.



Independent Evolution

The Employee Management API's database schema can change without redeploying the UI, as long as the contract holds.



Portability

The same API can serve a web client, a mobile client, or another backend service without modification.

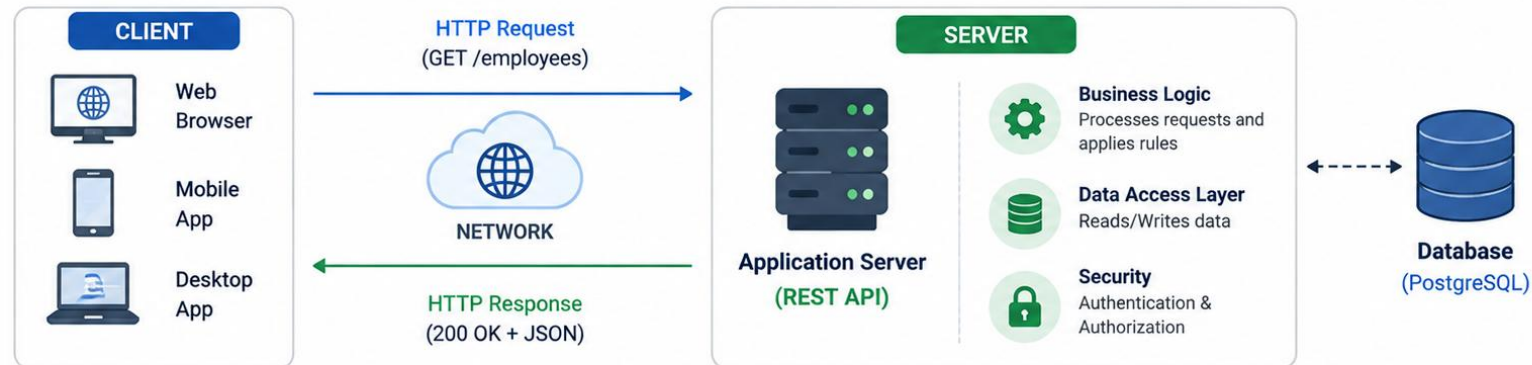
KEY TAKEAWAY Client-server separation is what lets the API team and the UI team ship on different schedules.

Client–Server Architecture

REST is built on a client–server model where the **client** initiates requests and the **server** provides responses containing the requested resources.



In REST, clients and servers are independent. They can evolve separately as long as the API contract remains consistent.



KEY POINTS



Separation of concerns

Client focuses on UI/UX, Server focuses on data and business logic.



Independent evolution

Client and server can be developed, deployed, and scaled independently.



Scalability

Multiple clients can access the same server, and servers can scale horizontally.



Interoperability

REST over HTTP allows communication across platforms and technologies.

STATELESS COMMUNICATION

Every request must carry all the context the server needs -- nothing is remembered between calls.



No Server-Side Session

Each HTTP request to /api/v1/employees must include everything needed to process it -- credentials, parameters, and body.



Authentication Every Time

A bearer token or API key travels on every call instead of relying on a login session stored server-side.



Enables Horizontal Scaling

Any server instance can handle any request because no instance holds client-specific state.

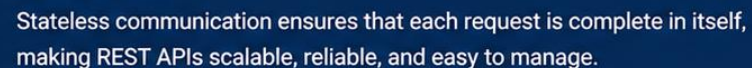
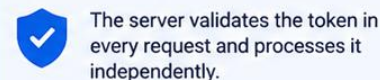
KEY TAKEAWAY If a request can only be understood by replaying prior requests, the API is not stateless -- and it will not scale.

The server **does not store** any client context between requests.

- **Scalability**
Easier to scale horizontally with multiple servers.
- **Reliability**
No session loss when a server goes down.
- **Simplicity**
Servers remain simpler with no session management.
- **Flexibility**
Any server can handle any request at any time.

The diagram illustrates the concept of session persistence. On the left, a computer icon represents the 'Client'. An arrow points from the client to a server icon, which is labeled 'Server'. The server is depicted as three stacked blue rectangles, each containing two green dots. A dashed arrow points from the server to a pink rounded rectangle labeled 'Session Store'. Above the server, red text reads 'Server stores client session state'.

- ✗ Server remembers client between requests.
- ✗ Requires sticky sessions.
- ✗ Harder to scale and maintain.



CACHEABILITY

Responses must explicitly declare whether -- and for how long -- they can be reused.



Responses Declare Freshness

Cache-Control, ETag, and Last-Modified headers tell clients and proxies how long a representation stays valid.



Cache the Right Things

A GET /api/v1/employees/{id} response can be cached until the record changes, cutting load on the service.



Explicit, Not Assumed

Uncacheable responses must say so (no-store) -- silence is not a safe default for sensitive employee data.

KEY TAKEAWAY Caching is a constraint the API must declare explicitly on every response, not a client-side guess.

Cacheability

REST responses define themselves as cacheable or non-cacheable using standard HTTP headers. Cacheable responses **reduce server load, improve performance**, and enhance scalability.

WHY CACHEABLE RESPONSES?



Better Performance
Faster responses for repeated requests.



Reduced Server Load
Fewer requests reach the server.

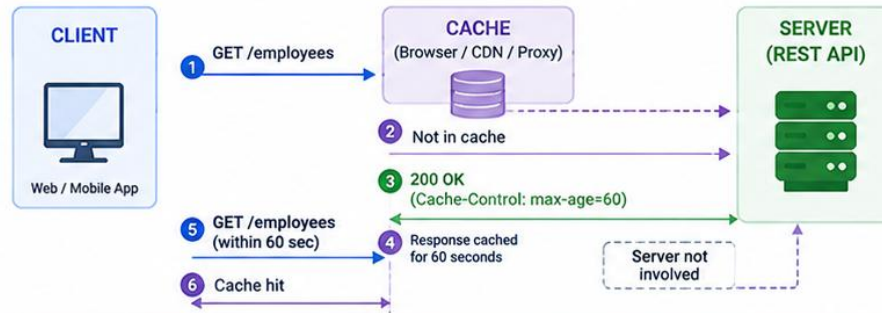


Scalability
More users can be served efficiently.



Lower Bandwidth Usage
Saves network and infrastructure costs.

HOW CACHEABILITY WORKS



EXAMPLE: CACHEABLE RESPONSE

```

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=60
ETag: "a1b2c3d4"
Last-Modified: Sat, 17 May 2025 10:30:00 GMT
{ "employees": [ ... ] }
  
```

- ✓ Can be cached by browsers, CDNs, and proxies.
- ✓ Served from cache until **max-age** expires.
- ✓ ETag / **Last-Modified** used for validation.

IMPORTANT HEADERS

Header	Purpose
Cache-Control	Directives for caching (e.g., max-age, no-cache, public, private, must-revalidate)
Expires	Date/time after which the response is considered stale
ETag	Entity tag for validating cached content
Last-Modified	Timestamp of last modification for the resource
Vary	Indicates which headers affect caching

EXAMPLE: NON-CACHEABLE RESPONSE

```

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
{ "message": "Request processed" }
  
```

- ✗ Must not be cached.
- ✗ Always fetched from the server.
- ✗ Used for sensitive or dynamic data.



KEY TAKEAWAY: Use cacheability to improve performance and scalability. Return cacheable responses for static or rarely changing data, and non-cacheable responses for sensitive or dynamic data.



GOOD PRACTICE
Cache GET responses for static or infrequently changing resources.



AVOID
Caching responses that contain user-specific or sensitive data.

UNIFORM INTERFACE

One consistent way to identify, manipulate, and interpret every resource in the API.



Resource Identification

Every resource (an employee, a department) has one URI -- /api/v1/employees/{id}.



Manipulation via Representations

Clients send and receive JSON representations; they never manipulate server-side objects directly.



Self-Descriptive Messages

Content-Type and status codes tell the client how to parse and interpret the response without out-of-band knowledge.



HATEOAS (Optional)

Responses may include links to related actions, letting clients discover what to do next instead of hardcoding URIs.

KEY TAKEAWAY Uniform interface is the constraint that makes one HTTP client library work against any RESTful API.

Uniform Interface

REST defines a uniform way for clients to interact with resources. This **simplicity** makes APIs easier to understand, use, and extend.

A uniform interface enables loose coupling between client and server.



KEY IDEA

All resources are identified by URIs and manipulated using a standard set of HTTP methods with uniform request/response formats (e.g., JSON).



BENEFITS

- Consistent and predictable API behavior
- Easier to learn and use
- Loosely coupled and scalable systems
- Enables independent evolution of client and server



GOOD PRACTICE

- Use nouns in URIs (resources, not actions)
- Use standard HTTP methods
- Use meaningful status codes
- Return hypermedia links when appropriate



AVOID

- RPC-style URLs (e.g., /getEmployee)
- Overloading HTTP methods
- Custom headers for common semantics
- Returning inconsistent response formats

THE FOUR ASPECTS OF UNIFORM INTERFACE

1



Resource Identification in Requests

Resources are identified using unique URIs.
Example:
`GET /employees/101`

2



Manipulation of Resources through Representations

Clients send/receive resource representations (e.g., JSON).
The server modifies the resource based on the representation.

3



Self-Descriptive Messages

Each message includes enough information to understand how to process it (headers, media type, status codes, etc.).

4



Hypermedia as the Engine of Application State (HATEOAS)

Responses can include links to related resources so the client can discover and navigate the API.

CLIENT



Request

`GET /employees/101`
Accept: application/json

Response

200 OK
Content-Type: application/json

EXAMPLE: MANAGING EMPLOYEE RESOURCE

```
{
  "id": 101,
  "name": "Jane Smith",
  "role": "Developer",
  "links": [
    { "rel": "self", "href": "/employees/101" },
    { "rel": "department", "href": "/departments/10" },
    { "rel": "update", "href": "/employees/101" },
    { "rel": "delete", "href": "/employees/101" }
  ]
}
```

FOLLOW-UP ACTIONS (USING SAME INTERFACE)

<code>GET</code>	<code>/employees</code>	List all employees
<code>GET</code>	<code>/employees/101</code>	Get employee details
<code>POST</code>	<code>/employees</code>	Create new employee
<code>PUT</code>	<code>/employees/101</code>	Replace employee
<code>PATCH</code>	<code>/employees/101</code>	Partial update
<code>DELETE</code>	<code>/employees/101</code>	Delete employee

LAYERED SYSTEM ARCHITECTURE

Clients should never need to know whether they're talking to the origin server or an intermediary.



Hidden Intermediaries

A gateway, load balancer, or cache can sit between the Angular client and the Spring Boot API without the client knowing.



Enforced at the Edge

Rate limiting, TLS termination, and authentication can live in a layer in front of the Employee Management API.



Independent Scaling

Each layer -- gateway, API, database -- scales on its own metrics without coordinated deployment.

KEY TAKEAWAY A well-layered API lets you insert a gateway or cache later without ever changing client code.

Layered System Architecture

REST allows a layered system architecture that separates concerns across multiple layers. Each layer has a **specific responsibility** and interacts only with adjacent layers.



KEY IDEA

Clients do not need to know how the server is implemented internally. Each layer can evolve independently without affecting other layers, as long as the API contract remains consistent.



BENEFITS

- Separation of concerns
- Improved maintainability
- Greater flexibility and scalability
- Enhanced security
- Technology independence

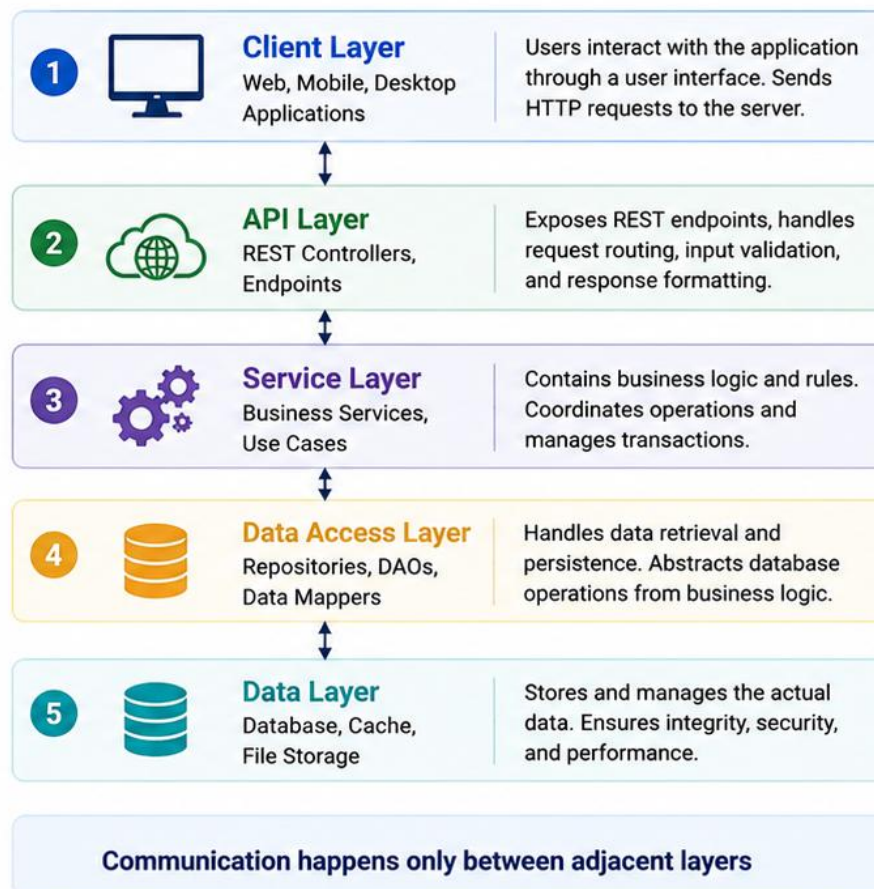


KEY TAKEAWAY

A layered architecture in REST APIs promotes clean separation of concerns, allowing systems to be more maintainable, scalable, and adaptable to change.

Top
(Closer to Client)

Bottom
(Closer to Data)



ARCHITECTURAL ADVANTAGES



Loose Coupling

Layers are independent and interact through well-defined interfaces.



Scalability

Each layer can be scaled independently based on its workload.



Security

Security controls can be applied at appropriate layers.



Maintainability

Changes in one layer do not impact other layers.



Technology Flexibility

Implementation details in one layer can change without affecting others.



EXAMPLE STACK

Angular / Mobile App → REST API → Spring Boot → JPA → PostgreSQL

RESOURCE-BASED API DESIGN

REST URIs name things; HTTP methods express what to do with them.



Nouns, Not Verbs

URIs name things (employees, departments), never actions (getEmployee, createEmployee).



Actions Live in HTTP Methods

The verb GET/POST/PUT/PATCH/DELETE expresses the action; the URI only ever names the resource.



Collections and Items

/api/v1/employees is a collection;
/api/v1/employees/{id} is a single item within it.

KEY TAKEAWAY If a URI has a verb in it, the design has drifted from REST toward RPC-style thinking.

Resource-Based API Design

REST APIs are built around **resources**—key business entities or concepts in your application.

Each resource is identified by a **unique URI** and is manipulated using standard **HTTP methods**.



KEY IDEA

Design your API by thinking about nouns (resources), not verbs (actions).

WHY RESOURCE-BASED?

- ✓ Aligns with how users think about the domain
- ✓ Leverages standard HTTP methods
- ✓ Makes APIs predictable and easy to use
- ✓ Enables better caching and scalability
- ✓ Promotes consistency across your API

EXAMPLE DOMAIN

Consider an **Employee Management System**.



Employee

An employee in the organization



Department

A department in the organization



Project

A project handled by employees



Leave Request

A leave request submitted by an employee

RESOURCE-BASED DESIGN PRINCIPLES



Identify Resources

Identify key business entities and concepts.



Use Nouns, Not Verbs

Use nouns in URIs to represent resources. Avoid action words.



Hierarchical Relationships

Use URI structure to represent relationships between resources.



Consistency

Use consistent naming, structure, and formatting across the API.

EXAMPLE RESOURCE MAPPING

Resource	Example URI	Description
Employees	<code>/employees</code>	Collection of all employees
Employee (by ID)	<code>/employees/{id}</code>	A single employee
Departments	<code>/departments</code>	Collection of departments
Department (by ID)	<code>/departments/{id}</code>	A single department
Projects	<code>/projects</code>	Collection of projects
Leave Requests	<code>/leave-requests</code>	Collection of leave requests
Leave Request (by ID)	<code>/leave-requests/{id}</code>	A single leave request



GOOD PRACTICES

- Use meaningful, plural nouns for collections.
- Use consistent, lowercase, hyphen-separated URIs.
- Use correct HTTP methods for operations.
- Use query parameters for filtering, sorting, and pagination.



AVOID

- Using verbs in URIs (e.g., `/getEmployees`, `/createEmployee`)
- Inconsistent naming (e.g., `/EmployeeList`, `/emp`)
- Overly complex or deeply nested URIs
- Mixing multiple operations in a single endpoint



KEY TAKEAWAY

A resource-based design creates intuitive, scalable, and maintainable APIs that follow REST principles and are easy for clients to consume.

IDENTIFYING API RESOURCES

Resources come from the business domain's nouns, not from database tables.



Start From the Domain

Employee, Department, and PerformanceReview are nouns an HR system already talks about -- they become resources directly.



Look for Relationships

An employee belongs to a department -- that becomes `/api/v1/departments/{id}/employees`, a nested resource.



Avoid Over-Fragmenting

Not every field needs its own resource -- salary is a property of Employee, not a separate endpoint.

KEY TAKEAWAY Good resource modeling starts with the domain's nouns, not with the database tables behind them.

Identifying API Resources

Resources are the nouns or business entities that your API exposes.

Identifying the right resources is the **foundation** of a well-designed REST API.



KEY IDEA

Think about the core business entities and concepts your application manages. Each of these becomes a resource in your API.

WHAT IS A RESOURCE?

A resource represents a business entity, concept, or data collection that can be:



Viewed
(GET)



Created
(POST)



Updated
(PUT/PATCH)



Deleted
(DELETE)

HOW TO IDENTIFY RESOURCES

1



Analyze Domain

Understand the domain and business processes. List key entities and concepts.

2



List Candidates

Create a list of nouns (entities) and important concepts.

3



Filter & Refine

Remove duplicates, synonyms, and terms that are not resources (verbs, actions).

4



Define Relationships

Identify how resources relate to each other (hierarchies, associations).

5



Validate & Prioritize

Validate with stakeholders and prioritize the most important resources.

EXAMPLE DOMAIN: EMPLOYEE MANAGEMENT SYSTEM

Domain Analysis

The system manages employees, departments, projects, and leave requests.

Key Concepts Identified

- Employee
- Department
- Project
- Leave Request
- Role
- Skill

Identified API Resources

Resource	Type
Employees	Collection / Entity
Departments	Collection / Entity
Projects	Collection / Entity
Leave Requests	Collection / Entity
Roles	Collection / Entity
Skills	Collection / Entity



GOOD PRACTICES

- Use business language (ubiquitous language).
- Model resources as nouns, not actions.
- Keep resources cohesive and independent.
- Align resources with domain boundaries.



AVOID

- Using verbs or actions as resources (e.g., /getEmployees).
- Exposing internal implementation details as resources.
- Creating too many or overly granular resources.
- Ignoring relationships between resources.



KEY TAKEAWAY

Well-identified resources lead to intuitive, consistent, and scalable REST APIs that are easy for clients to understand and consume.

RESOURCE URI DESIGN

A consistent naming convention, applied everywhere, is what makes an API predictable to use.



Plural Nouns

/employees, not /employee -- collections read naturally in the plural.



Lowercase, Hyphenated

/performance-reviews, not /performanceReviews or /PerformanceReviews -- consistent casing avoids confusion.



Hierarchy Reflects Ownership

/departments/{id}/employees expresses that employees belong to a department.



No Trailing Slashes or Extensions

/employees/42, never /employees/42/ or /employees/42.json -- the Accept header selects format.

KEY TAKEAWAY A consistent URI convention, applied everywhere, is worth more than any single 'clever' naming choice.



Resource URI Design

A well-designed URI makes your API intuitive, predictable, and easy to use.



DESIGN GUIDELINES



Use Nouns, Not Verbs

URIs should represent resources, not actions.



Use Plural Nouns

Represents a collection of resources consistently.



Use Hierarchical Structure

Show relationships using forward slashes.



Use Lowercase Letters

Avoid case sensitivity issues.



Use Query Parameters for Filtering, Sorting, Pagination

Keep the URI path clean and resource-focused.



Avoid File Extensions

Do not use .json, .xml, etc. Content negotiation handles format.



GOOD URI EXAMPLES

GET

/employees

Collection of employees

GET

/employees/123

Specific employee by ID

GET

/employees/123/departments

Departments of an employee

GET

/departments/10/employees

Employees in a department

GET

/employees?status=active&page=1&size=20

Filtering & pagination

GET

/employees?sort=lastName,asc

Sorting



BAD URI EXAMPLES

GET

/getEmployees

Uses verb

GET

/Employee/Details/123

Mixed case, uses verbs

GET

/getAllEmployeesList

Too verbose, uses verb

GET

/employees/getById/123

Action in URI

GET

/employees.json

File extension in URI

GET

/employees/123?getDepartments=true

Action as query parameter



Key Takeaway: A clean and consistent URI design improves usability, simplifies client integration, and future-proofs your API.

`</>` Well-designed URIs are the foundation of a great REST API.

GOOD VS BAD URI DESIGN

Every bad example below smuggles an action or an inconsistency into the URI itself.

Bad	Good
/getEmployee?id=42	GET /employees/42
/employees/42/deleteEmployee	DELETE /employees/42
/EmployeeList	GET /employees
/employees/42/update	PUT or PATCH /employees/42
/api/employee_department_list	GET /departments/{id}/employees

KEY TAKEAWAY Every 'bad' example smuggles an action into the URI -- the fix is always to move the verb into the HTTP method.

TRY IT YOURSELF — EXERCISE 1: DESIGN THE RESOURCE URI TABLE

Reinforces: resource-based design and the good-versus-bad URI slides you just saw, applied to the Northstar CRM Customer resource.

STEPS (notes/lab13-uri-table.md)

Step	What To Do
1 — Name the Nouns	List the CRM resources as plural nouns: customers, and the interactions beneath them.
2 — URI Table	Write the collection URI and the item URI, plus one sub-resource path.
3 — Fix a Bad URI	Rewrite /getCustomerById?id=CUS-1001 as a resource-oriented path.
4 — Scope	Design only — no controller code and no OpenAPI file yet.

URI shapes

```
collection    /api/customers
item          /api/customers/CUS-1001
sub-resource  /api/customers/CUS-1001/interactions
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-resource-uri-table.md (workspace: examples/module-13-exercises).



Good vs Bad URI Design

Well-designed URIs are consistent, intuitive, and resource-focused.
Avoid patterns that expose actions, implementation details, or complexity.



GOOD URI DESIGN

Resource-oriented, consistent, and easy to understand



Collection of employees

GET /employees



Single employee by ID

GET /employees/123



Departments of an employee

GET /employees/123/departments



Employees in a department

GET /departments/10/employees



Filtering & pagination

GET /employees?status=active
&page=1&size=20



Sorting

GET /employees?sort=lastName,asc



BAD URI DESIGN

Action-oriented, inconsistent, or implementation-focused



Uses verb in URI

GET /getEmployees



Mixed case

GET /Employee/Details/123



Too verbose / uses verb

GET /getAllEmployeesList



Action in URI

PUT /employees/123/update



File extension in URI

GET /employees.json



Action as query parameter

GET /employees/123?getDepartments=true



BEST PRACTICES



Focus on resources,
not actions.



Use nouns and
plural nouns.



Keep URIs short,
clean, and readable.



Use query parameters
for filtering, sorting,
and pagination.



A clean URI design improves API usability, simplifies client integration, and future-proofs your API.

HTTP REQUEST STRUCTURE

Every HTTP request has three parts: a request line, headers, and an optional body.



Request Line

METHOD /path?query HTTP/1.1
-- for example, GET
/api/v1/employees?department
=ENG HTTP/1.1.



Headers

Authorization, Content-Type,
and Accept describe who's
asking and what they want
back.



Body (Optional)

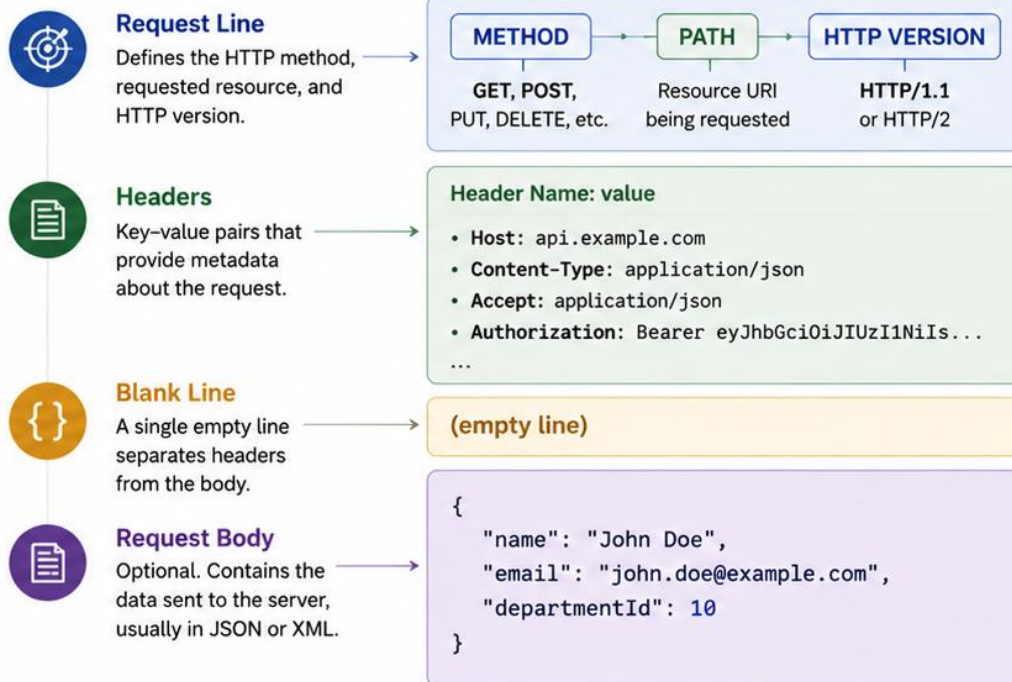
POST and PUT/PATCH carry a
JSON body; GET and DELETE
normally do not.

KEY TAKEAWAY A request's method and path say what and where; its headers say how; its body says with what data.



HTTP Request Structure

Every HTTP request sent by a client to the server follows a **standard structure**.



Tip: Well-formed HTTP requests ensure reliable communication between clients and servers.



Key Points

- **Request Line:** Specifies what you want to do and on which resource.
- **Headers:** Provide additional information like content type, authentication, caching, etc.
- **Blank Line:** Indicates the end of headers.
- **Body:** Carries the data for POST, PUT, PATCH requests.



Example: HTTP POST Request

```
POST /api/employees HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Length: 72

{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10
}
```

The example shows an HTTP POST request. The first four lines (Host, Content-Type, Accept, Authorization) are grouped under the label "Headers". The JSON object in the body is grouped under the label "Body (JSON)".

</> Understanding the HTTP request structure is the foundation of building and consuming REST APIs.

HTTP RESPONSE STRUCTURE

A response is judged first by its status code, then its headers, and finally its body.



Status Line

HTTP/1.1 201 Created -- the three-digit code is the first thing a client checks.



Headers

Content-Type, Location (on create), and caching headers describe the payload and how to treat it.



Body

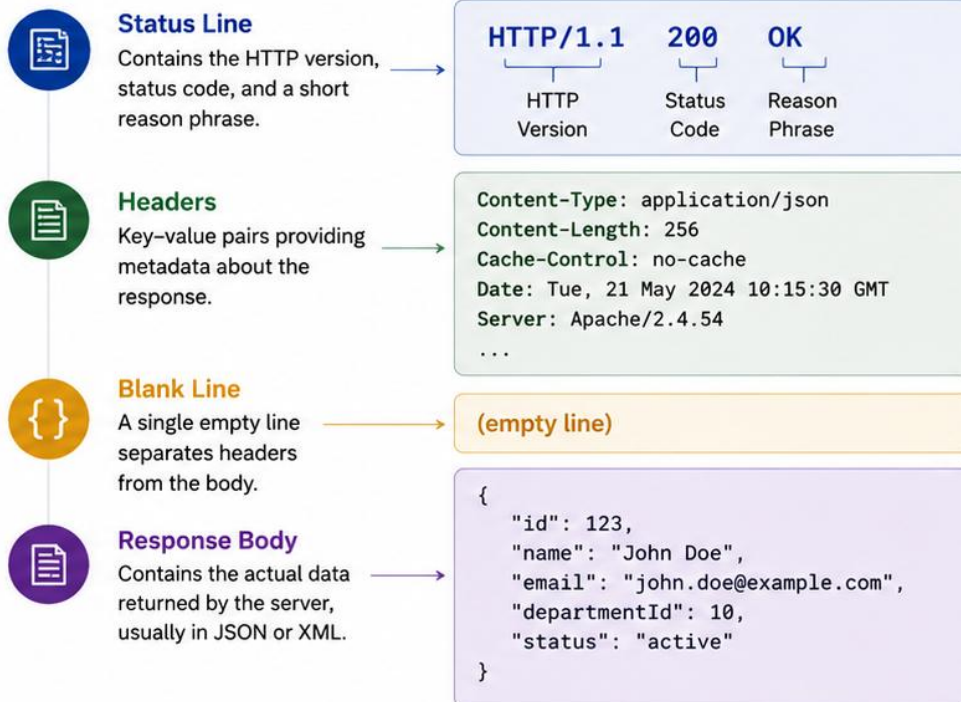
A JSON representation of the resource, or a structured error object when something went wrong.

KEY TAKEAWAY A response is judged first by its status code, then its headers, and only then by its body.



HTTP Response Structure

Every HTTP response returned by a server to a client follows a **standard structure**.



Tip: Always check the status code first. It determines how the client should interpret the response.



Key Points

- **Status Line:** Tells the client whether the request was successful or not.
- **Headers:** Provide details like content type, length, caching, authentication, rate limits, etc.
- **Blank Line:** Marks the end of headers.
- **Body:** Contains the response payload for successful responses.
- Not all responses have a body (e.g., 204 No Content).



Example: Successful HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 256
Cache-Control: no-cache
Date: Tue, 21 May 2024 10:15:30 GMT
Server: Apache/2.4.54

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "status": "active"
}
```

The example response is structured as follows:

- Headers:** The first six lines of the response (from `HTTP/1.1 200 OK` to `Server: Apache/2.4.54`).
- Body (JSON):** The JSON payload starting with `{` and ending with `}`.



A well-structured HTTP response ensures clarity, reliability, and better client-server communication.

HTTP METHODS OVERVIEW

Each HTTP method carries a specific, relied-upon behavior guarantee -- not just a naming convention.



Verbs Map to CRUD

GET reads, POST creates, PUT/PATCH update, DELETE removes -- each method has one clear job.



Semantics Are a Contract

Clients, proxies, and browsers all assume GET never changes data -- breaking that assumption breaks caching and retries.



Method Choice Is Design

Picking the right verb for each Employee Management API operation is as important as picking the right URI.

KEY TAKEAWAY HTTP methods are not interchangeable labels -- each one carries specific, relied-upon behavior guarantees.



HTTP Methods Overview

HTTP methods (verbs) define the **type of action** to be performed on a **resource**.

Method		Purpose	Safe	Idempotent	Typical Use Cases	Example Request
GET		Retrieve Retrieve a representation of a resource.	Yes	Yes	- Get employee by ID - List employees - Search / filter / paginate	GET /employees/123
POST		Create Create a new resource on the server.	No	No	- Create a new employee - Submit form data	POST /employees
PUT		Replace Replace an entire resource with new data.	No	Yes	- Update an employee completely - Replace resource state	PUT /employees/123
PATCH		Partial Update Partially update a resource with the provided data.	No	Yes	- Update specific fields - Partial profile update	PATCH /employees/123
DELETE		Delete Delete a resource from the server.	No	Yes	- Delete an employee - Remove a resource	DELETE /employees/123



Key Takeaway: Choose the right HTTP method to clearly express the intended action and maintain a consistent, predictable API.

`</>` Use the method that reflects the action, follow REST principles, and keep your API intuitive and consistent.

GET, POST, PUT, PATCH, AND DELETE

Five methods, five distinct jobs against the same /employees resource family.

Method	Purpose	Example
GET	Retrieve one or many employees	GET /employees/42
POST	Create a new employee under the collection	POST /employees
PUT	Replace an employee's full representation	PUT /employees/42
PATCH	Apply a partial update, e.g. salary only	PATCH /employees/42
DELETE	Remove or soft-delete an employee	DELETE /employees/42

KEY TAKEAWAY POST creates under a collection; PUT and PATCH act on an existing item -- full replace versus partial change.



GET – Retrieving Resources

The GET method is used to **retrieve data** from the server.

i GET requests should be **safe**, **idempotent**, and have no side effects on the server.

**Safe**

Does not modify
the resource

**Idempotent**

Same request
same result

**Cacheable**

Responses can be
cached by clients

**Read-Only**

Used only to
retrieve data



When to Use GET

- Retrieve a list of resources
- Retrieve a single resource by ID
- Search, filter, sort, and paginate data
- Get reference or lookup data
- Check the status of a resource



Example GET Request

```
GET /employees/123 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

No request body



Example Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 256
Cache-Control: no-cache

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "status": "active",
  "hireDate": "2024-01-15"
}
```



Query Parameters Examples

Purpose	Example Request	Description
Filtering	GET /employees?status=active	Get active employees
Pagination	GET /employees?page=1&size=20	Page 1, 20 records per page
Sorting	GET /employees?sort=lastName,asc	Sort by lastName ascending
Multiple Filters	GET /employees?deptId=10&status=active	Employees in dept 10 and active



Best Practices

- Use meaningful resource URIs.
- Use query parameters for filtering, sorting, pagination.
- Do not send sensitive data in query parameters.
- Return only necessary data.

**Tip:**

Always prefer GET for retrieving data. Avoid using GET for operations that change data on the server.



POST – Creating Resources

The POST method is used to **create a new resource** on the server.

i POST requests are **not idempotent** and may have side effects because they create new data.

When to Use POST

- Create a new resource
- Submit form data
- Create child resources
- Trigger server-side processes that result in a new resource



Request Body Guidelines

- ✓ Include only the data needed to create the resource.
- ✓ Do not send system-generated fields (id, createdAt, etc.).
- ✓ Validate data on the client and server.
- ✓ Use proper data types and formats.

Common Status Codes

Status Code	Meaning	When Used
201 Created	The resource was created successfully	On successful creation. Include Location header.
400 Bad Request	Invalid request or validation failed	When request data is invalid or incomplete
409 Conflict	Resource conflict (e.g., duplicate)	When resource already exists or conflict occurs
500 Internal Server Error	Server failed to create the resource	When an unexpected error occurs on the server



Creates New

Creates a new resource



Not Idempotent

Same request may create multiple resources



Not Cacheable

Responses should not be cached



Write Operation

Used to create data on the server

Example POST Request

```
POST /employees HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "hireDate": "2024-01-15"
}
```

Example Response (201 Created)

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /employees/123
Content-Length: 256

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "hireDate": "2024-01-15",
  "createdAt": "2024-05-21T10:30:00Z",
  "status": "active"
}
```

Best Practices

- ✓ Return 201 Created with the new resource representation.
- ✓ Include Location header with the URI of the new resource.
- ✓ Do not allow duplicate creation of unique resources.
- ✓ Keep request payloads small and focused.



Tip:

Use POST for creation operations only. Avoid mixing create and update logic.



PUT – Replacing Resources

The PUT method is used to **replace an entire existing resource** on the server.

i PUT is **idempotent**: sending the same request multiple times results in the same resource state.

**Idempotent**

Same request
same result

**Not Safe**

Modifies the
resource

**Not Cacheable**

Responses should
not be cached

**Replace Operation**

Replaces the entire
resource



When to Use PUT

- Update an existing resource completely
- Replace all fields of a resource
- Client knows the full state of the resource
- If the resource does not exist, server may create it (upsert behavior, depending on API design)



Key Characteristics

- ✓ Replaces the entire resource – all fields must be provided.
- ✓ Missing fields may be set to null or default values.
- ✓ Idempotent – same request produces same result.
- ✓ Useful when the client has the complete latest representation.



Common Status Codes

Status Code	Meaning	When Used
200 OK	Resource replaced successfully	When the resource existed and was updated
201 Created	Resource created successfully	When the resource did not exist and was created
400 Bad Request	Invalid request or validation failed	When request payload is invalid
404 Not Found	Resource not found	When replacing is not allowed and resource doesn't exist
500 Internal Server Error	Server error	When an unexpected error occurs



Example PUT Request

```
PUT /employees/123 HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "hireDate": "2024-01-15",
  "status": "inactive"
}
```



Example Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 256
Cache-Control: no-cache

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "hireDate": "2024-01-15",
  "status": "inactive",
  "updatedAt": "2024-05-21T10:30:00Z"
}
```



Best Practices

- ✓ Use PUT for full replacement, not partial updates.
- ✓ Include all required fields in the request body.
- ✓ Use PATCH for partial updates.
- ✓ Validate the entire object on the server.
- ✓ Return the updated resource in the response.

**Tip:**

If the client only wants to update a few fields, use PATCH instead of PUT to avoid unintentionally overwriting other fields.



PUT replaces the target resource with the provided representation. Use it when you have the complete updated data.

PATCH – Partial Updates

Update only the fields that need to change.



What is PATCH?

PATCH is used to apply partial modifications to an existing resource. Only the fields provided in the request are updated. Fields not included in the request remain unchanged.



Key Benefit: Efficient updates when you only need to modify a few fields of a resource.



When to Use PATCH

- When updating one or more fields of a resource.
- When the resource is large and sending the entire object (PUT) is unnecessary.
- When the client does not have (or does not want to send) the complete representation.



Example – Update Employee Email

PATCH Request

```
{  
  "email": "john.doe@company.com",  
  "phone": "+1-555-987-6543"  
}
```

Updated Resource (Response)

```
{  
  "id": 101,  
  "name": "John Doe",  
  "email": "john.doe@company.com",  
  "phone": "+1-555-987-6543",  
  "department": "Engineering",  
  "position": "Developer"  
}
```



Only the fields provided in the PATCH request (*email*, *phone*) are updated. Other fields (*name*, *department*, *position*) remain unchanged.



Best Practices



Be Specific

Update only the fields that need to change. Avoid sending unnecessary data.



Define Patch Format

Use a consistent format (JSON Merge Patch or JSON Patch) across your API.



Validate Input

Ensure incoming fields are valid and allowed to be updated.



Secure Updates

Authorize users to update only the fields they are allowed to modify.

★ **Remember:** PATCH is not inherently idempotent. Repeating the same PATCH request may produce different results depending on the server logic.

DELETE – Removing Resources

Remove a resource identified by its URI.



What is DELETE?

DELETE is used to remove a resource on the server. After a successful DELETE, the resource is no longer available at that URI.

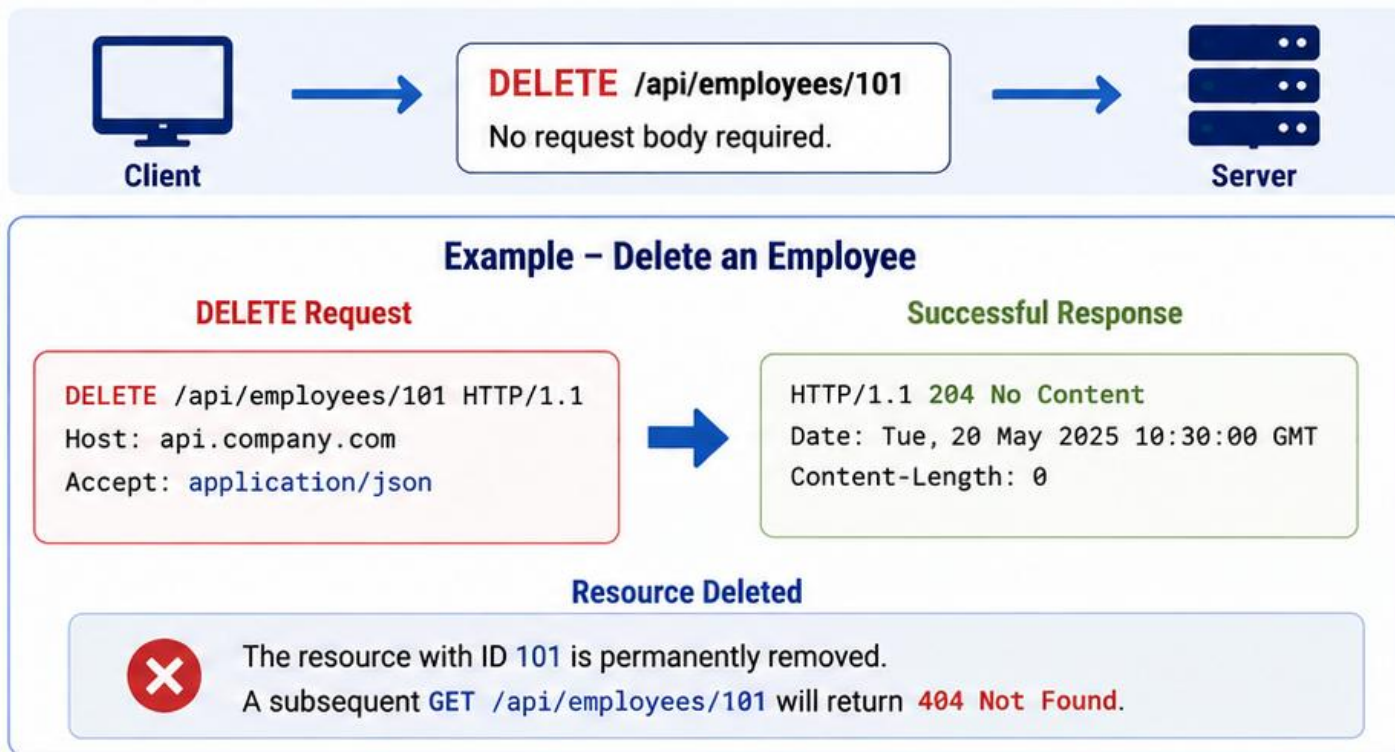


Key Benefit: Helps keep your data clean and ensures resources that are no longer needed are permanently removed.



When to Use DELETE

- When a resource is no longer needed.
- When the user explicitly requests removal.
- When performing cleanup or data lifecycle management.



Common DELETE Responses

204

No Content

Resource deleted successfully. No response body.

200

OK

Resource deleted successfully. Optional response body.

404

Not Found

Resource does not exist or has already been removed.

403

Forbidden

You do not have permission to delete the resource.

409

Conflict

Resource cannot be deleted due to a business rule.

500

Server Error

Server failed to process the delete request.



Best Practices



Be Idempotent

Multiple DELETE requests on the same resource should have the same effect.



Secure Deletions

Validate permissions before deleting. Use role-based access control.



Consider Soft Deletes

If needed, implement soft delete (e.g., is_deleted flag) instead of permanently removing records.



Audit Deletions

Log who deleted what and when for audit and compliance.



Confirm Before Delete

Consider requiring confirmation for destructive operations in the client UI.



Remember: DELETE removes resources. Use it carefully and ensure proper authorization, validation, and auditing.

SAFE VS IDEMPOTENT OPERATIONS

Safe means no side effects at all; idempotent means repeating the call is harmless.

Method	Safe?	Idempotent?
GET	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
PATCH	No	No (usually)
POST	No	No

KEY TAKEAWAY Idempotent means repeating the call is harmless; safe means the call never changes server state at all.

Safe vs Idempotent Operations

Understanding the behavior of HTTP methods for reliable API design.

HTTP Methods Quick Reference

GET

Retrieve

PUT

Replace

DELETE

Remove

POST

Create

PATCH

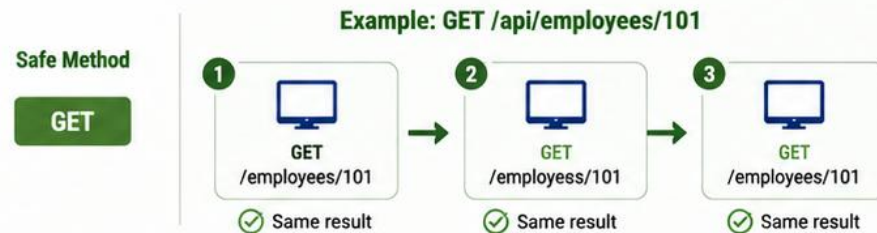
Partial Update



SAFE OPERATIONS

Operations that are read-only and do not modify the server state.

- They can be performed multiple times without changing the state.
- Repeated requests have the same effect as a single request.



Safe = No side effects on the server state.



IDEMPOTENT OPERATIONS

Operations that can be performed multiple times with the same effect.

- The server state after one request is the same as after many identical requests.
- These operations may change the state, but repeated requests do not.

Idempotent Methods

PUT**DELETE****PATCH**

Example: PUT /api/employees/101



Idempotent = Same final state, no matter how many times it is called.

Operation Type	Definition	Examples (HTTP Methods)	Multiple Calls Result	Modifies Server State?	Safe?	Idempotent?
Safe	Read-only operations that do not change server state.	GET	Same result every time.	No	Yes	Yes
Idempotent	Can be called multiple times with the same effect.	PUT, DELETE, PATCH	Same final state every time.	Yes (but result is consistent)	No	Yes
Neither Safe Nor Idempotent	May change state and repeated calls have different effects.	POST	Different result each time (e.g., creates multiple resources).	Yes	No	No



Key Takeaways

- Use **GET** for retrieving data (Safe & Idempotent).
- Use **PUT, DELETE, PATCH** for updates/removals (Idempotent).
- Use **POST** for creating new resources (Neither Safe nor Idempotent).
- Idempotency helps in handling retries and network failures safely.

★ **Reminder:** Safety is about not changing data. Idempotency is about the consistency of the final state.



Design your APIs with predictable and reliable behavior.

HTTP STATUS CODE CATEGORIES

The first digit of a status code tells a client, at a glance, what kind of outcome occurred.

Range	Category	Meaning
1xx	Informational	Request received, processing continues (rarely used directly)
2xx	Success	The request was received, understood, and accepted
3xx	Redirection	Further action is needed to complete the request
4xx	Client Error	The request is invalid or cannot be fulfilled as sent
5xx	Server Error	The server failed to fulfill a valid request

KEY TAKEAWAY The first digit of a status code alone tells a client whether to retry, fix the request, or give up.

HTTP Status Code Categories

HTTP status codes are three-digit responses from the server that indicate the outcome of a request.



The first digit of the status code defines the category and the expected type of outcome.

1xx – Informational

The request was received, continuing process.



Common Codes

- 100** Continue
- 101** Switching Protocols
- 102** Processing (WebDAV)
- 103** Early Hints

Example Use

Tells the client to continue the request or switch protocol.

2xx – Success

The request was successfully received, understood, and accepted.



Common Codes

- 200** OK
- 201** Created
- 204** No Content
- 206** Partial Content

Example Use

Indicates successful operation or resource creation.

3xx – Redirection

Further action is needed to complete the request.



Common Codes

- 301** Moved Permanently
- 302** Found (Temporary)
- 304** Not Modified
- 307** Temporary Redirect
- 308** Permanent Redirect

Example Use

Instructs the client to take a different action or URL.

4xx – Client Error

The request contains bad syntax or cannot be fulfilled.



Common Codes

- 400** Bad Request
- 401** Unauthorized
- 403** Forbidden
- 404** Not Found
- 409** Conflict
- 422** Unprocessable Entity

Example Use

Indicates errors caused by the client.

5xx – Server Error

The server failed to fulfill a valid request.



Common Codes

- 500** Internal Server Error
- 501** Not Implemented
- 502** Bad Gateway
- 503** Service Unavailable
- 504** Gateway Timeout

Example Use

Indicates errors on the server side.



Key Takeaways



Understand the Category

The first digit defines the type of response.



Use Appropriate Codes

Return the most accurate code for the situation.



Be Consistent

Follow standard usage across your API.



Improve Client Experience

Meaningful codes help clients handle responses correctly.



Remember: Use the right status code to communicate clearly, improve reliability, and make your APIs easy to consume.

2XX, 4XX, AND 5XX RESPONSES

The specific code within each category tells a client exactly how to react.



2xx Success

200 OK, 201 Created, 204 No Content -- the request worked; 201 also returns a Location header.



4xx Client Error

400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict -- the caller must fix something.



5xx Server Error

500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable -- the caller did nothing wrong.

KEY TAKEAWAY 4xx says 'you need to change the request'; 5xx says 'we need to fix the server' -- never confuse the two.

2xx Success Responses

2xx status codes indicate that the request was successfully received, understood, and accepted.



A 2xx response means the operation completed successfully on the server.

Status Code	Name	Meaning	Common Use Cases
200	OK	The request was successful. The server has returned the requested data.	- GET request success - PUT or PATCH success - Any successful retrieval
201	Created	The request was successful and a new resource was created.	- POST request success - New resource creation
204	No Content	The request was successful, but there is no content to return.	- DELETE success - PUT/PATCH success when no content is returned
206	Partial Content	The server is returning partial content as requested by the client.	- Range requests (GET with Range header) - Large file streaming
207	Multi-Status	The request was successful, but multiple status messages are being returned.	- WebDAV operations - Batch or multi-resource operations

Example Responses

200 OK

GET /api/employees/101

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@company.com",
  "department": "Engineering"
}
```

201 Created

POST /api/employees

```
{
  "id": 102,
  "name": "Jane Smith",
  "email": "jane.smith@company.com"
}
```

Location: /api/employees/102

204 No Content

DELETE /api/employees/101

(No response body)

206 Partial Content

GET /api/files/report.pdf

Content-Range: bytes 0-1023/5000
Content-Length: 1024
(Partial binary content...)



Tip: Always choose the most appropriate 2xx code to communicate the exact outcome of the operation to the client.



Best Practices



Be Specific

Use the most precise 2xx status code that matches the outcome.



Use Location Header

For 201 Created, return the Location header with the URI of the new resource.



Avoid Unnecessary Body

Use 204 No Content when there's nothing meaningful to return.



Consistency

Be consistent in how you structure and document successful responses.



Document Clearly

Document all possible 2xx responses in your API specification.



Remember: Success is not just about 200 OK. Use the right 2xx code to make your API clear, predictable, and easy to integrate.

4xx Client Errors

4xx status codes indicate that the client made a request that contains bad syntax or cannot be fulfilled.



4xx errors are caused by the client and should be **fixed by the client**. The server understands the request but refuses to process it.

Status Code	Name	Meaning	Common Use Cases
400	Bad Request 	The server cannot or will not process the request due to something that is perceived to be a client error.	- Malformed JSON - Invalid request syntax - Missing required parameters
401	Unauthorized 	Authentication is required and has failed or has not been provided.	- Missing or invalid token - Expired credentials - Unauthenticated access
403	Forbidden 	The server understood the request but refuses to authorize it.	- Insufficient permissions - Access to restricted resource - Role-based access violation
404	Not Found 	The requested resource could not be found on the server.	- Invalid URI - Resource does not exist
409	Conflict 	The request conflicts with the current state of the resource.	- Duplicate resource - Version conflict - Business rule violation
422	Unprocessable Entity 	The server understands the request but cannot process the contained instructions.	- Validation errors - Semantic errors in request - Business rule failure
429	Too Many Requests 	The client has sent too many requests in a given amount of time.	- Rate limiting - Throttling protection

Example Error Responses

400 Bad Request POST /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Invalid JSON format",
  "path": "/api/employees"
}
```

401 Unauthorized GET /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 401,
  "error": "Unauthorized",
  "message": "Missing or invalid authentication token",
  "path": "/api/employees"
}
```

403 Forbidden GET /api/admin/reports

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 403,
  "error": "Forbidden",
  "message": "You do not have permission to access this resource",
  "path": "/api/admin/reports"
}
```

404 Not Found GET /api/employees/999

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Employee not found",
  "path": "/api/employees/999"
}
```

409 Conflict PUT /api/employees/101

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Email already exists",
  "path": "/api/employees/101"
}
```

422 Unprocessable Entity POST /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Validation failed",
  "details": ["email is invalid"],
  "path": "/api/employees"
}
```

429 Too Many Requests GET /api/search

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 429,
  "error": "Too Many Requests",
  "message": "Rate limit: 1000 requests per hour",
  "path": "/api/search"
}
```



Retry-After: 60



Best Practices



Be Specific

Return the most accurate 4xx code that reflects the actual problem.



Provide Helpful Messages

Include clear and user-friendly messages to help clients understand the issue.



Stay Consistent

Use a consistent error response structure across your API.



Avoid Information Leakage

Do not expose sensitive information in error responses.



Respect Rate Limits

Use 429 to protect your system from abuse and ensure stability.



Remember: 4xx errors are the client's responsibility. Return the right code and guidance so clients can fix the request.

5xx Server Errors

5xx status codes indicate that the server failed to fulfill a valid request. These errors are not the client's fault.



5xx errors are caused by the **server side**. They indicate unexpected conditions, failures, or inability to process the request.

Status Code	Name	Meaning	Common Causes
500	Internal Server Error 	A generic error occurred on the server. The server encountered an unexpected condition.	- Uncaught exceptions - Null pointer or runtime errors - Misconfiguration
501	Not Implemented 	The server does not support the functionality required to fulfill the request.	- HTTP method not supported - Feature not implemented
502	Bad Gateway 	The server, while acting as a gateway or proxy, received an invalid response from upstream server.	- Invalid response from upstream - Network issues - Timeout from upstream
503	Service Unavailable 	The server is currently unable to handle the request. It may be overloaded or down for maintenance.	- Server overload - Maintenance mode - Resource exhaustion
504	Gateway Timeout 	The server, while acting as a gateway or proxy, did not receive a timely response from upstream server.	- Upstream timeout - Slow backend service - Network latency
505	HTTP Version Not Supported 	The server does not support the HTTP protocol version used in the request.	- Unsupported HTTP version - Protocol mismatch

Example Error Responses

500 Internal Server Error

GET /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred.",
  "path": "/api/employees"
}
```

502 Bad Gateway

GET /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 502,
  "error": "Bad Gateway",
  "message": "Upstream service returned an invalid response.",
  "path": "/api/employees"
}
```

503 Service Unavailable

GET /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 503,
  "error": "Service Unavailable",
  "message": "The server is currently unavailable. Please try again later.",
  "path": "/api/employees",
  "retryAfter": 30
}
```

504 Gateway Timeout

GET /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 504,
  "error": "Gateway Timeout",
  "message": "Upstream service did not respond in time.",
  "path": "/api/employees"
}
```

501 Not Implemented

TRACE /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 501,
  "error": "Not Implemented",
  "message": "The server does not support this method.",
  "path": "/api/employees"
}
```

505 HTTP Version Not Supported

HTTP/2.0 /api/employees

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "status": 505,
  "error": "HTTP Version Not Supported",
  "message": "The HTTP version used is not supported.",
  "path": "/api/employees"
}
```



Tip: Always log 5xx errors with correlation IDs and monitor them. Never expose internal details in production error responses.

Example Log

```
ERROR [reqId=0f3d2a] 2025-05-20T10:30:00Z -
Failed to process request /api/employees - NullPointerException
```



Best Practices



Log with Context

Include timestamps, request IDs, user info, and stack traces in logs.



User-Friendly Messages

Return generic messages to clients. Avoid exposing sensitive details.



Monitor & Alert

Set up monitoring and alerts for 5xx errors.



Graceful Degradation

Design systems to degrade gracefully when dependencies fail.



Retry When Appropriate

Clients may retry for 502, 503, and 504 with backoff.

★ **Remember:** 5xx errors are on the server. Fix the root cause, improve robustness, and ensure high availability.

TRY IT YOURSELF — EXERCISE 2: MAP METHODS TO STATUS CODES

Reinforces: the HTTP method, safety, idempotence, and status-code slides you just covered, mapped onto the Customer resource.

STEPS (notes/lab13-method-status-map.md)

Step	What To Do
1 — Method Table	For each URI, list the methods allowed and what each one does.
2 — Success Codes	Give the exact success status for each: 200, 201 with Location, or 204.
3 — Failure Codes	Map CUS-9999 to 404 and a malformed body to 400.
4 — Safe vs Idempotent	Mark each method safe, idempotent, both, or neither.

Method and status map

```
GET    /api/customers/{id}  -> 200 | 404    safe + idempotent
POST   /api/customers       -> 201 + Location neither
PUT    /api/customers/{id} -> 200 | 404    idempotent, not safe
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-method-status-map.md (workspace: examples/module-13-exercises).

REQUEST AND RESPONSE JSON

One consistent JSON shape and naming convention, used identically across every endpoint.



One Format, Both Directions

The Employee Management API accepts and returns application/json for every endpoint in this module.



Field Naming Convention

camelCase field names (employeeId, hireDate) match Java field naming and Jackson's default (de)serialization.



Nesting Reflects Structure

A department field nests { "id": ..., "name": ... } rather than flattening into departmentId/departmentName .

KEY TAKEAWAY Pick one JSON naming convention for the whole API and never mix camelCase and snake_case across endpoints.

Request and Response JSON

JSON (JavaScript Object Notation) is the most common format for exchanging data in REST APIs.



Why JSON?

- ✓ Lightweight and human-readable
- ✓ Language independent
- ✓ Easy to parse and generate

Example: HTTP Request with JSON Body

POST

/api/employees

Content-Type: application/json

Request Body

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@company.com",
  "phone": "+1-555-123-4567",
  "department": {
    "id": 10,
    "name": "Engineering"
  },
  "hireDate": "2025-05-20",
  "salary": 85000.50,
  "skills": ["Java", "Spring Boot", "REST APIs"],
  "active": true
}
```

Key Points

- Keys are strings in double quotes.
- String values use double quotes.
- Numbers, booleans do not use quotes.
- Objects are enclosed in { }.
- Arrays are enclosed in [].
- Commas separate fields.

Example: HTTP Response with JSON Body

200 OK

Content-Type: application/json

Response Body

```
{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@company.com",
  "phone": "+1-555-123-4567",
  "department": {
    "id": 10,
    "name": "Engineering"
  },
  "hireDate": "2025-05-20",
  "salary": 85000.50,
  "skills": ["Java", "Spring Boot", "REST APIs"],
  "active": true,
  "createdAt": "2025-05-20T10:15:30Z",
  "updatedAt": "2025-05-20T10:15:30Z",
  "links": {
    "self": "/api/employees/101",
    "department": "/api/departments/10"
  }
}
```

Key Points

- Server returns the representation of the resource.
- Includes generated fields (id, timestamps).
- Nested objects and arrays are supported.
- Links help with navigation (HATEOAS).

Common Data Types in JSON

- Aa** String
"text value"
- 123** Number
123 or 123.45
- ✓** Boolean
true or false
- []** Array
["item1", "item2"]
- { }** Object
{"key": "value"}
- null** Null
null

Validation Example



If the request JSON is invalid, the server returns a **400 Bad Request** with error details.

```
{
  "timestamp": "2025-05-20T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Invalid JSON format",
  "path": "/api/employees"
}
```



Best Practices



Use Descriptive Keys

Use meaningful and consistent naming conventions.



Keep It Consistent

Maintain consistent structure across requests and responses.



Avoid Nulls

Prefer omitting fields over sending null values.



Validate Always

Validate input JSON on the server before processing.



Remember: Well-structured JSON leads to clean APIs, fewer errors, and better developer experience.

CONTENT-TYPE AND ACCEPT HEADERS

Content-Type describes what was sent; Accept describes what the client is willing to receive.



Content-Type Describes the Body

A POST with a JSON payload must send Content-Type: application/json so the server parses it correctly.



Accept Requests a Format

The client's Accept: application/json header tells the server which representation it can understand.



415 on Mismatch

An unsupported Content-Type returns 415 Unsupported Media Type instead of guessing at the body.

KEY TAKEAWAY Content-Type describes what you sent; Accept describes what you're willing to receive back.



Content-Type and Accept Headers

HTTP headers Content-Type and Accept are used in REST APIs to define the format of the data being sent in the request and the format the client expects in the response.



Content-Type Header

- Specifies the media type of the request body.
- Informs the server how to interpret the data.
- Common value: `application/json`



Accept Header

- Specifies the media type the client expects in the response.
- Helps the server return the data in the desired format.
- Common value: `application/json`

→ HTTP REQUEST (Example)

```
POST /api/employees HTTP/1.1
Host: api.example.com
Content-Type: application/json # Tells server request body is JSON
Accept: application/json # Tells server client expects JSON
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "departmentId": 10
}
```

← HTTP RESPONSE (Example)

```
HTTP/1.1 201 Created
Content-Type: application/json
Date: Tue, 07 May 2024 10:15:30 GMT

{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "createdAt": "2024-05-07T10:15:30Z"
}
```

★ BEST PRACTICES



Always set Content-Type when sending a request body.



Always set Accept to define the response format you expect.



Use specific media types (e.g., `application/json`) instead of wildcards when possible.



Consistent use improves compatibility and API reliability.



KEY TAKEAWAY: `Content-Type` tells the server how to read your request. `Accept` tells the server how you want the response.

API ERROR RESPONSE DESIGN

One consistent JSON error shape lets every client handle every failure the same way.

Field	Purpose
timestamp	When the error occurred
status	The HTTP status code, repeated in the body
error	Short machine-readable error category, e.g. VALIDATION_ERROR
message	Human-readable explanation of what went wrong
path	The request path that failed, e.g. /api/v1/employees/42
traceld	Correlation ID for tracing this request across logs and services

KEY TAKEAWAY A consistent error body shape lets every client handle failures with one parsing routine, not one per endpoint.



API Error Response Design

A well-designed error response helps clients understand what went wrong, why it happened, and how to fix it.

DESIGN PRINCIPLES



Consistent

Use the same structure for all error responses.



Informative

Provide clear, actionable information.



Secure

Do not expose internal system details or stack traces.



Machine-Readable

Use structured fields that clients can parse reliably.



Traceable

Include a correlation or trace ID for support and debugging.

STANDARD ERROR RESPONSE (EXAMPLE)

```
{
  "timestamp": "2024-05-07T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for one or more fields.",
  "path": "/api/employees",
  "traceId": "d4f8b9a0ee2d34f6c",
  "errors": [
    {
      "field": "email",
      "rejectedValue": "john.doe",
      "message": "Email must be a valid format"
    },
    {
      "field": "age",
      "rejectedValue": -5,
      "message": "Age must be greater than 0"
    }
  ]
}
```

When the error occurred (UTC time).

HTTP status code.

Short, standard description of the error.

Human-readable message explaining the error.

The API endpoint where the error occurred.

Unique ID to trace the request across systems.

Detailed validation or business errors (optional).

USE THE RIGHT HTTP STATUS

400

Bad Request

Invalid request or validation error.

401

Unauthorized

Authentication required or failed.

403

Forbidden

Authenticated but not allowed to access.

404

Not Found

Resource not found.

409

Conflict

Resource state conflict (e.g., duplicate email).

422

Unprocessable Entity

Semantically correct request but business rule violation.

500

Internal Server Error

Unexpected server error.

BEST PRACTICES



Keep messages user-friendly and actionable.



Return field-level errors for validation issues.



Avoid exposing stack traces, SQL errors, or internal IDs.



Always include a trace/correlation ID for support.

EXAMPLES OF DIFFERENT ERROR RESPONSES

401 Unauthorized

```
{
  "timestamp": "2024-05-07T10:15:30Z",
  "status": 401,
  "error": "Unauthorized",
  "message": "Invalid or expired token.",
  "path": "/api/employees"
}
```

Authentication failure

404 Not Found

```
{
  "timestamp": "2024-05-07T10:15:30Z",
  "status": 404,
  "error": "Not Found",
  "message": "Employee with id 999 not found.",
  "path": "/api/employees/999"
}
```

Resource not found

500 Internal Server Error

```
{
  "timestamp": "2024-05-07T10:15:30Z",
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred.",
  "path": "/api/employees"
}
```

Server-side error



KEY TAKEAWAY: Design error responses that are consistent, informative, secure, and traceable to improve API usability and debugging.

TRY IT YOURSELF — EXERCISE 3: WRITE THE ERROR CONTRACT

Reinforces: the JSON payload and API error response design slides you just saw, fixed as one error shape every endpoint returns.

STEPS (notes/lab13-error-contract.md)

Step	What To Do
1 — Error Fields	Define the fields every error returns: timestamp, status, code, message, path.
2 — Correlation	Add a correlation identifier and say which header carries it.
3 — Two Examples	Write the 404 body for CUS-9999 and the 400 body for a validation failure.
4 — Leak Check	State what an error must never contain: stack traces, SQL, internal hostnames.

Error envelope

```
{ "timestamp": "...", "status": 404, "code": "CUSTOMER_NOT_FOUND",  
  "message": "No customer with id CUS-9999", "path": "/api/customers/CUS-9999",  
  "correlationId": "lab-request-001" }      // no stack traces, ever
```

TRY IT NOW — Complete Exercise 3 before continuing: `exercises/exercise-03-error-contract.md` (workspace: `examples/module-13-exercises`).

PAGINATION, FILTERING, AND SORTING

Query parameters on the same collection endpoint, not new endpoints, control how it's returned.



Pagination

GET

/employees?page=0&size=20
returns a bounded page plus
totalElements, so large HR
datasets never load in one
response.



Filtering

GET

/employees?department=ENG&
status=ACTIVE narrows the
collection using query
parameters, not new endpoints.



Sorting

GET

/employees?sort=lastName,asc
orders results without the client
re-sorting thousands of records
itself.

KEY TAKEAWAY Pagination, filtering, and sorting are all expressed as query parameters on the same collection endpoint.



Pagination

Pagination breaks a large dataset into smaller chunks (pages) so clients can load data efficiently and navigate through results.

WHY PAGINATION?



- Improves performance by limiting the amount of data returned per request.
- Reduces bandwidth and memory usage.
- Enhances user experience with faster load times.

COMMON APPROACH



- Client requests a specific page with a page size (limit).
- Server returns the data slice plus pagination metadata.
- Client uses metadata to navigate (next/previous pages).

TWO COMMON PARAMETERS



- page** : Page number (1-based)
- size** : Number of items per page

→ HTTP REQUEST (Example)

```
GET /api/employees?page=2&size=10 HTTP/1.1
Host: api.example.com
Accept: application/json
```

← HTTP RESPONSE (Example)

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "content": [
    {
      "id": 11,
      "firstName": "Alice", "lastName": "Johnson",
      "department": "Engineering",
    },
    {
      "id": 12,
      "firstName": "Smice", "department": "Finance"
    }
  ],
  "page": {
    "number": 2,
    "size": 10,
    "totalElements": 95,
    "totalPages": 10,
    "first": false,
    "last": false,
    "hasNext": true,
    "hasPrevious": true
  }
}
```

Data for the requested page
(2nd page, 10 items)

Pagination metadata

BEST PRACTICES



Set a maximum page size to prevent large requests (e.g., max 100).



Validate page and size to handle invalid values gracefully.



Use consistent parameter names (e.g., page, size).



Consider sorting results to ensure consistent paging.

PAGINATION METADATA EXPLAINED

number	Current page number (0 or 1-based depending on API)
size	Number of items per page
totalElements	Total number of items across all pages
totalPages	Total number of pages
first	Is this the first page?
last	Is this the last page?
hasNext	Is there a next page?
hasPrevious	Is there a previous page?

NAVIGATING BETWEEN PAGES

First Page Current Page Next Page
page=1&size=10 page=2&size=10 page=3&size=10

Always use the metadata to build pagination controls dynamically.



KEY TAKEAWAY: Pagination helps APIs scale. Always return both the data slice and clear metadata so clients can navigate efficiently.



Filtering

Filtering allows clients to request only the data that matches specific conditions, reducing payload size and improving performance.

WHY FILTERING?



- Returns only relevant data.
- Reduces bandwidth and response size.
- Improves performance for large datasets.
- Enhances user experience with targeted results.

COMMON APPROACH



- Clients send filter criteria as query parameters.
- Server applies the filters and returns matching results.
- Multiple filters can be combined.

COMMON FILTER OPERATORS

eq	Equal (e.g., ?name=John)
ne	Not equal (e.g., ?status!=INACTIVE)
gt	Greater than (e.g., ?salary>50000)
lt	Less than (e.g., ?salary<50000)
gte	Greater than or equal (e.g., ?age>=30)
lte	Less than or equal (e.g., ?age<=30)
like	Pattern match (e.g., ?name=Jo% or ?name=%son)
in	Belongs to a list (e.g., ?status=ACTIVE, INACTIVE)

→ HTTP REQUEST (Example)

```
GET /api/employees?  
department=Engineering&status=ACTIVE&salary>50000&name=Jo%&page=1&size=5  
HTTP/1.1  
Host: api.example.com  
Accept: application/json
```

Applied Filters

- department = Engineering
- status = ACTIVE
- salary > 50000
- name starts with 'Jo'
- page = 1, size = 5



← HTTP RESPONSE (Example)

```
HTTP/1.1 200 OK  
Content-Type: application/json  
{  
  "content": [  
    {  
      "id": 101,  
      "name": "John Doe",  
      "department": "Engineering",  
      "salary": 75000,  
      "status": "ACTIVE"  
    },  
    {  
      "id": 104,  
      "name": "Jonathan Smith",  
      "department": "Engineering",  
      "salary": 62000,  
      "status": "ACTIVE"  
    }  
  ],  
  "page": {  
    "number": 1,  
    "size": 5,  
    "totalElements": 23,  
    "totalPages": 5,  
    "first": true,  
    "last": false,  
    "hasNext": true,  
    "hasPrevious": false  
  }  
}
```

Filtered
data slice
(5 items)

Pagination
metadata

BEST PRACTICES



Use clear and consistent parameter names.



Validate filter inputs to prevent invalid queries.



Document supported filters and operators.



Apply indexes on commonly filtered fields.

COMMON FILTERING PATTERNS

Single Filter	GET /api/employees?status=ACTIVE
Multiple Filters	GET /api/employees?dept=Engineering&status=ACTIVE
Range Filter	GET /api/employees?salary>50000&salary<100000
Like / Search	GET /api/employees?name=Jo%
IN Filter	GET /api/employees?status=ACTIVE, INACTIVE
Date Range	GET /api/employees?createdDate>=2024-01-01&createdDate<=2024-12-31
Pagination + Filter	GET /api/employees?status=ACTIVE&page=1&size=10

TIPS



- Avoid exposing internal field names in filters.
- Limit maximum page size to prevent abuse (e.g., max 100).
- Combine filtering with sorting and pagination for best results.



KEY TAKEAWAY:

Filtering helps APIs return only the data clients need. Use clear parameters, validate inputs, and combine with pagination for efficient and scalable API design.



Filter +



Sort +



Paginate



Sorting

Sorting allows clients to specify the order in which results are returned based on one or more fields, making data easier to read and analyze.

WHY SORTING?



- Improves data readability.
- Helps users quickly find relevant information.
- Reduces client-side processing.
- Enhances performance for large datasets.

COMMON APPROACH



- Client specifies sort fields and direction (ascending or descending).
- Server returns results in the requested order.

SORT PARAMETERS



- sort** : Field name(s) to sort by
- order** : Sort direction (asc or desc)
- Default** : If not provided, use a sensible default (e.g., createdAt desc)

→ HTTP REQUEST (Example)

```
GET /api/employees?sort=lastName,asc&sort=salary,desc HTTP/1.1
Host: api.example.com
Accept: application/json
```

Applied Sorting

- First sort by `lastName` in ascending order (A → Z)
- For employees with the same `lastName`, sort by `salary` in descending order (high → low)



← HTTP RESPONSE (Example)

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "content": [
    {"id": 101, "firstName": "Alice", "lastName": "Anderson", "salary": 95000},
    {"id": 102, "firstName": "Bob", "lastName": "Anderson", "salary": 88000},
    {"id": 103, "firstName": "Charlie", "lastName": "Brown", "salary": 105000},
    {"id": 104, "firstName": "David", "lastName": "Clark", "salary": 72000},
    {"id": 105, "firstName": "Eva", "lastName": "Clark", "salary": 65000}
  ],
  "page": { "number": 0, "size": 5, "totalElements": 42 }
}
```

SORT DIRECTION OPTIONS

Value	Description
asc	Ascending order (A → Z, 0 → 9, oldest → newest)
desc	Descending order (Z → A, 9 → 0, newest → oldest)
(default)	API-defined default if order is not provided (e.g., desc for createdAt)

EXAMPLES OF SORTING

Request	Description
GET /api/employees?sort=lastName,asc	Sort by lastName A → Z
GET /api/employees?sort=salary,desc	Sort by salary high → low
GET /api/employees?sort=lastName,asc&sort=firstName,asc	Sort by lastName, then firstName
GET /api/employees?sort=createdAt,desc	Sort by newest first
GET /api/employees	No sorting (use default)

BEST PRACTICES



Allow sorting only on safe and indexed fields to protect performance.



Document supported fields and directions clearly.



Combine sorting with pagination to avoid large payloads.



Validate sort fields and return a clear error for invalid values.

TIP



Multiple sort fields are applied in the order they are provided. The server sorts by the first field, then the next, and so on.



KEY TAKEAWAY: Sorting helps APIs return data in the **right** order. Use clear parameters, validate inputs, and combine with filtering and pagination for powerful and user-friendly APIs.



Filter +



Sort +



Paginate =



Better Results

API VERSIONING STRATEGIES

A breaking change needs a way to reach only the clients ready for it -- versioning is that mechanism.



URI Versioning

/api/v1/employees -- the version is visible and cacheable, the most common enterprise choice.



Header-Based Versioning

A custom header or Accept media type carries the version, keeping URIs stable.



Query Parameter Versioning

?version=1 is simple but easy to omit accidentally and rarely used in production APIs.

KEY TAKEAWAY Pick one versioning strategy before the first client integrates -- changing it later breaks every existing consumer.



API Versioning Strategies

API versioning allows you to evolve your API over time without breaking existing clients. Choose a strategy that fits your organization's needs and long-term maintainability.

1. URI VERSIONING



Version is included in the request URI.
Most common and easy to understand.

EXAMPLE REQUEST

```
GET /api/v1/employees
Host: api.example.com
```

PROS

- Simple and explicit
- Easy to cache
- Widely adopted

CONS

- URI becomes longer
- Multiple versions visible in the API

2. HEADER-BASED VERSIONING



Version is passed in a custom request header.
Keeps URIs clean and stable.

EXAMPLE REQUEST

```
GET /api/employees
Host: api.example.com
X-API-Version: 1
```

PROS

- Clean and consistent URIs
- Easy to change version without URI changes
- Useful when versioning multiple resources

CONS

- Harder to test with browsers
- Caching can be complex
- Not as visible as URI versioning

3. QUERY PARAMETER VERSIONING



Version is passed as a query parameter.
Flexible but less common.

EXAMPLE REQUEST

```
GET /api/employees?version=1
Host: api.example.com
```

PROS

- Easy to implement
- Works with existing infrastructure
- No custom headers needed

CONS

- Less clean URIs
- Can be overlooked
- Not as RESTful

VERSIONING BEST PRACTICES



Version only when necessary.
Avoid versioning for minor changes that don't break clients.



Plan for backward compatibility.
Avoid breaking changes in existing versions.



Deprecate old versions gradually.
Provide clear deprecation notices and timelines.



Communicate changes clearly.
Maintain a changelog for each version.



Use consistent versioning strategy across all APIs.

STRATEGY COMPARISON

Strategy	Example	Visibility	Caching	Ease of Use	Best For
URI Versioning	/api/v1/employees	High	Easy	Easy	Public APIs, simplest approach
Header-Based	X-API-Version: 1	Medium	Moderate	Moderate	Internal APIs, clean URIs
Query Parameter	/api/employees?version=1	Low	Easy	Easy	Simple use cases, prototypes

DEPRECATION EXAMPLE

```
HTTP/1.1 299 Deprecated
Deprecation: true
Sunset: Wed, 31 Dec 2025 23:59:59 GMT
Link: </api/v2/employees>; rel="successor-version"
{
  "message": "This API version is deprecated and
  will be removed on 2025-12-31."
}
```



KEY TAKEAWAY: Version your APIs to evolve safely over time. Choose a strategy that balances **clarity**, **usability**, and **maintainability** for both your team and your API consumers.



Choose Strategy



Implement Consistently



Evolve Safely

URI VERSIONING VS HEADER-BASED VERSIONING

Two ways to carry a version, weighed head to head for the Employee Management API's own choice.

Approach	Example	Trade-off
URI Versioning	/api/v1/employees	Simple, visible, cacheable; URI changes on every major version
Header-Based	Accept: application/vnd.company.v1+json	Stable URIs; harder to test in a browser and less discoverable

KEY TAKEAWAY This module standardizes on URI versioning (/api/v1/...) for the Employee Management API because it is explicit and cache-friendly.



URI Versioning

URI Versioning includes the version in the request path. It is the most common and straightforward way to version REST APIs.

WHAT IS URI VERSIONING?



The API version is part of the URI path.

Each version has its own set of endpoints.

Example pattern:

```
/api/v{version}/resource
```

EXAMPLE

Version 1

GET `/api/v1/employees`

GET `/api/v1/employees/101`

Version 2

GET `/api/v2/employees`

GET `/api/v2/employees/101`



Both versions can exist and be supported independently.

HOW IT WORKS

1

Client includes the version in the URI (e.g., `/api/v1/...`).

2

Server routes the request to the appropriate versioned controller.

3

Each version can evolve independently.

4

Older versions remain available for existing clients.

PROS AND CONS

PROS

- ✓ Very clear and easy to understand.
- ✓ Easy to implement and test.
- ✓ Works well with caching and API gateways.
- ✓ Multiple versions can coexist without conflicts.

CONS

- ✗ URIs become longer.
- ✗ More endpoints to maintain.
- ✗ May expose internal versioning strategy to clients.

SPRING BOOT EXAMPLE

Controller for v1

```
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeV1Controller {

    @GetMapping
    public List<EmployeeV1Dto> getAll() {
        // v1 implementation
    }
}
```

Controller for v2

```
@RestController
@RequestMapping("/api/v2/employees")
public class EmployeeV2Controller {

    @GetMapping
    public List<EmployeeV2Dto> getAll() {
        // v2 implementation (enhanced)
    }
}
```

BEST PRACTICES



Version only when there are backward-incompatible changes.



Document each version clearly.



Deprecate old versions with clear timelines.



Ensure consistent versioning across all APIs.



KEY TAKEAWAY:

URI versioning is simple, visible, and widely used. Use it when **clarity** and **stability** across versions are important.

EXAMPLE FULL REQUEST

```
GET /api/v2/employees/101 HTTP/1.1
Host: api.example.com
Accept: application/json
```



Clearly indicates the API version in the path.





Header-Based Versioning

Header-Based Versioning passes the API version in a custom request header. It keeps URIs clean and provides flexibility for evolving APIs.

WHAT IS HEADER-BASED VERSIONING?



- ✓ The API version is sent in a custom HTTP header.
- ✓ URIs remain clean and consistent.
- ✓ Useful when you want to avoid exposing versions in the URL.
- ✓ Example header: `X-API-Version`

HOW IT WORKS

- 1 Client includes the version in a custom header (e.g., `X-API-Version: 2`).
- 2 Server reads the header and routes the request to the appropriate versioned controller.
- 3 Each version can evolve independently.
- 4 Older versions remain available for existing clients.

EXAMPLE REQUEST

```
GET /api/employees HTTP/1.1
Host: api.example.com
Accept: application/json
X-API-Version: 2
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

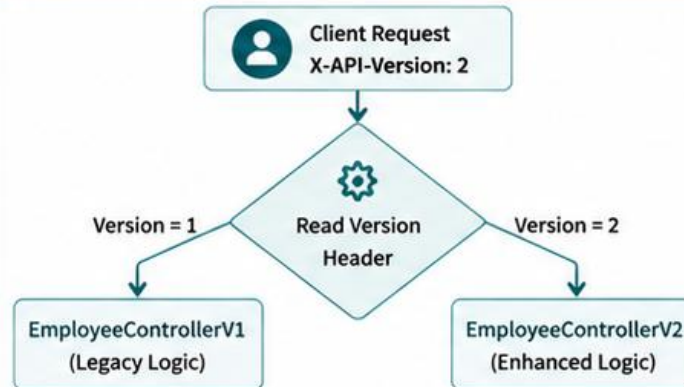
Custom header carries the API version.

SERVER SIDE EXAMPLE (SPRING BOOT)

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @GetMapping
    public List<EmployeeDto> getEmployees(
        @RequestHeader(name = "X-API-Version",
            required = false, defaultValue = "1") int version) {
        if (version == 2) {
            return getEmployeesV2();
        } else {
            return getEmployeesV1();
        }
    }
}
```

VERSION ROUTING EXAMPLE



PROS AND CONS

PROS

- ✓ Keeps URIs clean and stable.
- ✓ Flexible and easy to change.
- ✓ Works well with existing infrastructure and proxies.
- ✓ Can be enforced through API gateways.

CONS

- ✗ Less visible to API consumers in browser or docs.
- ✗ Harder to test manually without tools.
- ✗ Requires proper documentation and client awareness.

BEST PRACTICES



- Use a clear and consistent header name (e.g., `X-API-Version`).
- Document supported versions and their timelines.
- Provide a default version if the header is missing.
- Deprecate old versions with clear notifications.

EXAMPLE RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
X-API-Version: 2
X-Deprecated: false
```

→ Echo the version in the response for transparency.



KEY TAKEAWAY

Header-Based Versioning keeps your API clean and flexible while allowing you to evolve safely. Great for enterprise APIs and internal systems.

TRY IT YOURSELF — EXERCISE 4: PLAN COLLECTIONS AND VERSIONING

Reinforces: the pagination, filtering, sorting, and versioning-strategy slides you just covered, decided for the Customer collection.

STEPS (notes/lab13-collection-versioning.md)

Step	What To Do
1 — Query Parameters	Name the page, size, sort, and filter parameters and their defaults.
2 — Response Envelope	Say what the list response returns besides the rows themselves.
3 — Version Choice	Pick URI or header versioning, and give the reason you chose it.
4 — Breaking Change	Name one change that forces a new version and one that does not.

Collection query design

```
GET /api/v1/customers?page=0&size=20&sort=name,asc&status=ACTIVE
-> { content:[...], page:0, size:20, totalElements:42, totalPages:3 }
breaking: remove/rename a field    non-breaking: add an optional field
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-collection-and-versioning.md (workspace: examples/module-13-exercises).

INTRODUCTION TO OPENAPI

OpenAPI turns an API's shape into a document both humans and tools can read and validate against.



Machine-Readable Contract

OpenAPI (formerly Swagger) describes every path, method, parameter, and schema in a YAML or JSON document.



Generates Docs and Clients

Tools turn the same OpenAPI file into interactive documentation and generated client SDKs.



Contract Before Code

Teams can review and agree on the API shape before a single controller is written.

KEY TAKEAWAY OpenAPI turns the API's shape into a document that both humans and tools can read and validate against.

Introduction to OpenAPI



OpenAPI (formerly Swagger) is a **standard**, language-agnostic **specification for describing REST APIs**.



Standardized API Contracts

Defines a machine-readable contract for your API, ensuring consistency and clarity.



Better Collaboration

Provides a single source of truth for backend developers, frontend developers, testers, and DevOps teams.



Automation Friendly

Enables automatic generation of documentation, client SDKs, server stubs, and test cases.



Improved API Quality

Encourages consistent design, validation, and versioning of APIs.

```
1 openapi: 3.0.3
2 info:
3   title: Employee Management API
4   version: 1.0.0
5   description: API for managing employees
6 servers:
7   - url: https://api.example.com/v1
8 paths:
9   /employees:
10    get:
11      summary: Get all employees
12      responses:
13        '200':
14          description: Successful response
15          content:
16            application/json: {}
```



Design

Define your API contract



Build

Implement and validate APIs



Document

Auto-generate interactive docs



Consume

Generate client SDKs and test



Key Takeaway

OpenAPI helps you design, document, and maintain APIs that are predictable, discoverable, and easy to integrate.



OPENAPI DOCUMENT STRUCTURE

Four top-level sections describe an entire API: info, servers, paths, and components.



info

Title, version, and description of the API as a whole.



servers

The base URLs where the API is actually reachable, e.g.
<https://api.company.com>.



paths

Every URI and the operations (GET, POST, ...) available on it.



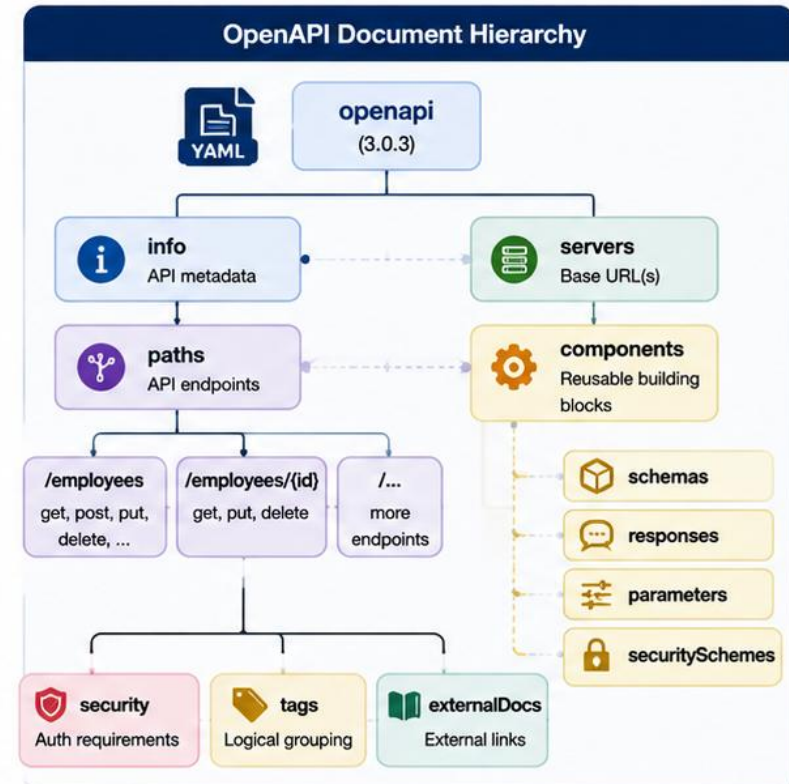
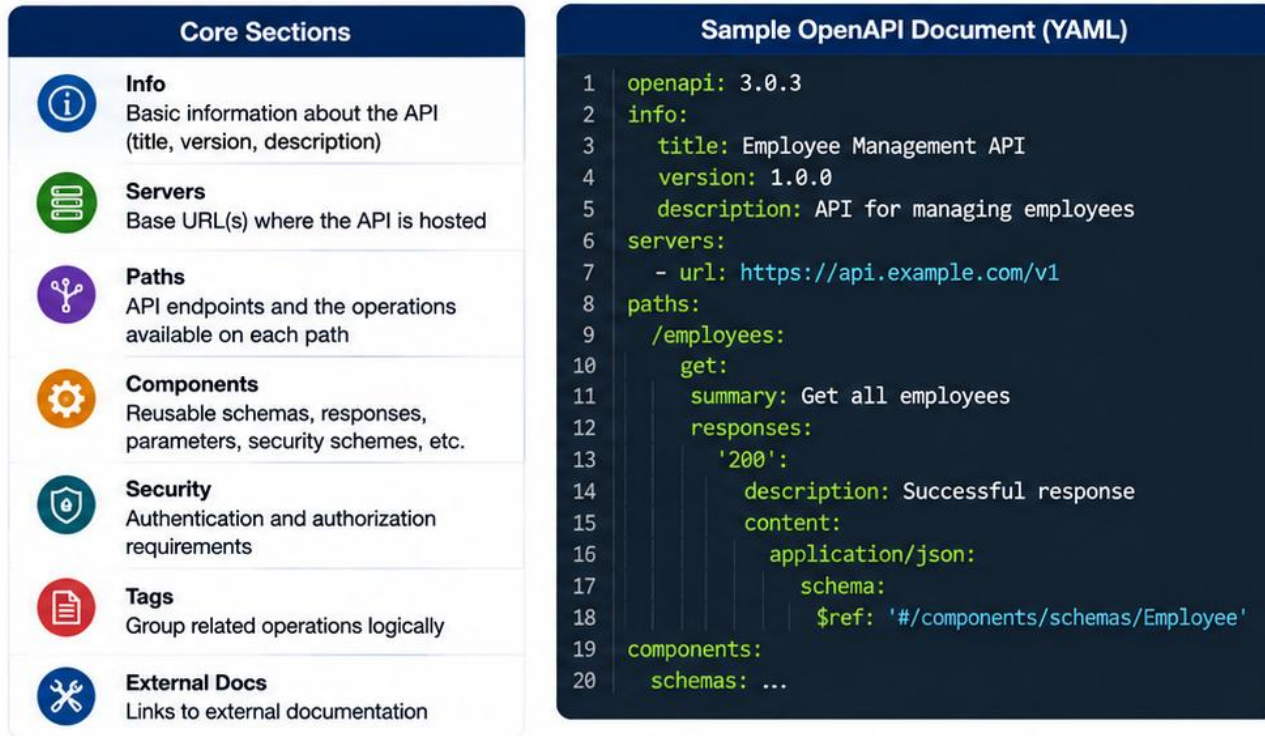
components

Reusable schemas, like the Employee and Department objects, referenced from multiple paths.

KEY TAKEAWAY components/schemas is where reuse pays off -- define Employee once, reference it from every operation that needs it.

OpenAPI Document Structure

An **OpenAPI** document (in YAML or JSON) describes your entire API in a machine-readable format.



Key Takeaway

The OpenAPI document is the single source of truth for your API.
It drives documentation, client SDKs, testing, and contract-first development.



PATHS AND OPERATIONS

A path is a noun in the document; each operation under it is one verb applied to that noun.



Path Defines the Resource

/employees/{employeeId}
declares the URI template,
including its path parameter.



Operation Defines the Verb

Under that path, get, post, put,
patch, and delete each describe
one HTTP method's behavior.



Operation Metadata

summary, description, and
operationId make each
operation discoverable in
generated docs and SDKs.

KEY TAKEAWAY In OpenAPI, a path is a noun and each operation under it is one verb applied to that noun.

Paths and Operations

In **OpenAPI**, **paths** define the API endpoints and **operations** specify the supported HTTP methods and their details.

{...} OpenAPI YAML (Excerpt)

```
1 openapi: 3.0.3
2 info:
3   title: Employee Management API
4   version: 1.0.0
5 servers:
6   - url: https://api.example.com/v1
7 paths:
8   /employees:
9     get:
10      summary: Get all employees
11      operationId: getEmployees
12      responses:
13        '200':
14          description: Successful response
15      post:
16        summary: Create a new employee
17        operationId: createEmployee
18        responses:
19          '201':
20            description: Employee created
21 /employees/{id}:
22   get:      # Get employee by ID
23   put:      # Replace employee
24   patch:    # Partial update
25   delete:  # Delete employee
```

/employees
Collection
Endpoint

/employees/{id}
Individual
Resource
Endpoint

Key Concepts



Path
Represents a URL
endpoint (resource).



Operation
Represents an HTTP
method on a path.



Operation Details
Include summary, parameters,
request/response schemas,
and status codes.

Example: /employees/{id} Operations

Method	Purpose	Idempotent	Safe	Typical Response
GET	Retrieve an employee by ID	✓	✓	200 OK
PUT	Replace the entire employee	✓	✗	200 OK / 204 No Content
PATCH	Partially update the employee	✓	✗	200 OK
DELETE	Delete the employee	✓	✗	204 No Content

Best Practices



- Keep paths consistent and resource-focused.
- Use nouns, not verbs, in paths.
- Group related operations under the same path.



Provide clear summaries and
accurate responses for each
operation.



**OpenAPI
Document**



Paths

Define endpoints



Operations

Define HTTP methods



Details

Parameters, schemas,
responses, status codes



Consumers

Use for documentation,
SDKs, and testing

REQUEST AND RESPONSE SCHEMAS

A create request and its response almost never share exactly the same shape.



Request Schema

Describes the JSON body an operation accepts, e.g. EmployeeCreateRequest with firstName, lastName, departmentId.



Response Schema

Describes the JSON body returned, e.g. EmployeeResponse, often including fields the request never sent, like id.



Never the Same Shape

A create request rarely mirrors its response -- fields like id and hireDate only appear in the response.

KEY TAKEAWAY Separate request and response schemas because the server always knows more than the client sent.

Request Schemas

Request schemas define the **structure**, **data types**, and **validation rules** for the data sent to your API.



Key Concepts

Schema
Describes the shape of the request body using JSON Schema.

Validation
Enforces required fields, data types, formats, and constraints.

Reuse
Schemas are defined once and reused across multiple operations.

Clarity
Clear schemas improve API reliability and developer experience.

Example: POST /employees Request Schema

POST /employees

Content-Type: application/json

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "jobTitle": "Software Engineer",
  "hireDate": "2025-05-01",
  "salary": 85000.00,
  "isActive": true
}
```

Schema (YAML)

```
components:
  schemas:
    EmployeeCreateRequest:
      type: object
      required:
        - firstName
        - lastName
        - email
      properties:
        firstName:
          type: string
          minLength: 1
          maxLength: 50
        lastName:
          type: string
          minLength: 1
          maxLength: 50
        ... (other fields)
```

Common JSON Schema Types	
Type	Description
string	Text value
number	Integer or floating-point number
integer	Whole number
boolean	true or false
array	List of values
object	Key-value structure
null	Null value

Validation Keywords (Examples)	
Keyword	Description
required	Marks a field as mandatory
minLength / maxLength	Limits string length
minimum / maximum	Numeric range constraints
pattern	Regex pattern for strings
format	Predefined formats (email, date, uuid, etc.)

Where Request Schemas Are Used

Request Bodies
POST, PUT, PATCH operations

Query Parameters
Complex objects in query string

Header Parameters
Custom headers with structured data

Reusable Components
Shared schemas across multiple APIs

Best Practices

- Use meaningful property names.
- Mark required fields explicitly.
- Provide constraints and formats.
- Reuse schemas with \$ref.
- Keep schemas simple and consistent.

Response Schemas



Response schemas define the **structure and data types** of the data returned by your API, helping ensure **consistency and clarity**.

Key Concepts



Schema

Defines the shape of the response body using JSON Schema.



Accurate Responses

Clear response schemas reduce ambiguity and improve client implementation.



Reusable

Schemas can be reused across multiple endpoints and status codes.



Complete Coverage

Define schemas for success, error, and edge-case responses.

Example: GET /employees/{id} Response Schemas

200 OKContent-Type: `application/json`

EmployeeResponse Schema (JSON)

```
{
  "id": 1001,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": {
    "id": 10,
    "name": "Engineering"
  },
  "salary": 85000.00,
  "hireDate": "2025-05-01",
  "isActive": true
}
```

Schema (YAML)

```
type: object
properties:
  id:
    type: integer
    format: int64
  firstName:
    type: string
  lastName:
    type: string
  email:
    type: string
    format: email
  department:
    type: object
    $ref: '#/components/schemas/Department'
  salary:
    type: number
    format: double
  hireDate:
    type: string
    format: date
  isActive:
    type: boolean
required: [id, firstName, lastName, email]
```

Common Response Status Codes

Status Code	Purpose	Schema Example
200 OK	Successful request	EmployeeResponse
201 Created	Resource created	EmployeeResponse
400 Bad Request	Invalid request	ErrorResponse
404 Not Found	Resource not found	ErrorResponse
500 Internal Server Error	Server error	ErrorResponse

Example: ErrorResponse Schema (JSON)

```
{
  "timestamp": "2025-05-01T12:45:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Employee with id 1001 not found",
  "path": "/employees/1001"
}
```

Where Response Schemas Are Used



Response Bodies

Define structure for successful or error responses.



Different Status Codes

Provide specific schemas for each relevant HTTP status code.



Pagination / Lists

Define array structures and pagination metadata in responses.



Reusable Components

Use `$ref` to reuse schemas and keep definitions consistent.

Best Practices



- Use consistent naming and data types.
- Include all possible response codes.
- Provide meaningful error messages.
- Reuse schemas with `$ref`.
- Keep responses simple and focused.



API-FIRST DEVELOPMENT VS CODE-FIRST

Write the contract first, or generate the contract from the code -- both are legitimate, for different teams.

Approach	How It Works	Best For
API-First	Write the OpenAPI contract first, generate stubs, then implement	Teams building clients and servers in parallel
Code-First	Write Spring controllers first, generate OpenAPI from annotations	Small teams iterating quickly on one codebase

KEY TAKEAWAY API-first pays off the moment more than one team -- frontend, backend, partners -- needs to agree on the contract early.

API-First Development

API-First means designing your API contract **before writing a single line of code**.
The API becomes the primary artifact that drives development and collaboration.

Key Benefits



Better Collaboration

Provides a clear contract that aligns backend, frontend, and other teams.



Early Feedback

Stakeholders can review and provide feedback on the API before implementation.



Parallel Development

Teams can build clients and servers independently using the contract.



Consistency & Governance

Enforces standards, reuse, and versioning across the organization.



Reduced Rework

Well-defined APIs upfront minimize changes and integration issues.

API-First Development Lifecycle

1



Design the API Contract

Define endpoints, requests, responses, schemas, rules, and errors using OpenAPI.

2



Review & Validate

Stakeholders review the contract for completeness, consistency, and usability.

3



Generate Artifacts

Generate server stubs, client SDKs, documentation, and test cases from the contract.

4



Implement the API

Implement the backend based on the contract. Use contract tests to ensure compliance.

5



Test & Validate

Run contract tests and integration tests to validate behavior.

6



Publish & Govern

Publish the API, version it, and govern changes through the lifecycle.

Example: Define the API First



openapi: 3.0.3

info:

title: Employee Management API
version: 1.0.0

paths:

/employees:

get:

summary: Get all employees

responses:

'200':

description: Successful response

content:

application/json:

schema:

\$ref: '#/components/schemas/Employee'



Popular API-First Tools



Swagger
Editor



Stoplight
Studio



Postman



Insomnia



Redocly



Key Takeaway

API-First Development puts the API contract at the center of your delivery process, enabling quality, speed, and alignment across the entire enterprise.



API
Contract



Tests



Client SDKs



Server
Implementation



Documentation

Code-First vs API-First

Both approaches can deliver great APIs. The right choice depends on your **project goals**, **team structure**, and the need for **collaboration and governance**.

Code-First Approach

Build the API by writing code first, then document.



1. Write Code

Implement endpoints in the application.



2. Run Application

Verify API behavior and functionality.



3. Generate Docs

Use tools to generate OpenAPI documentation.



Best Suited For

- Internal APIs
- Small teams
- Rapid prototyping
- Projects with fewer external dependencies



Challenges

- Contract is not defined upfront
- Harder to get early feedback
- Higher risk of breaking changes
- Documentation may lag behind

VS

API-First Approach

Design and publish the API contract first, then build.



1. Design Contract

Define endpoints, schemas, rules, and responses in OpenAPI.



2. Review & Collaborate

Stakeholders review and provide feedback early.



3. Generate Stubs & SDKs

Generate server stubs, client SDKs, and docs from the contract.



4. Implement & Test

Implement the API, test, and publish.



Best Suited For

- Public APIs
- Large or distributed teams
- Strong governance environments
- API products and platforms



Challenges

- Requires more upfront effort
- Needs strong collaboration
- Design mistakes early can impact rework

Comparison at a Glance

Aspect	Code-First	API-First
Starting Point	Write code	Design contract
Documentation	Generated after implementation	Created upfront
Stakeholder Feedback	Late in the process	Early and continuous
Contract Clarity	May evolve during development	Defined and agreed upfront
Best For	Internal, simple, fast iterations	External, complex, long-term, governed APIs
Risk of Breaking Changes	Higher	Lower
Time to Market	Faster initially	Slightly slower initially, faster overall

When to Choose What?



Choose Code-First when:

You need speed, the API is internal, and requirements are likely to change.



Choose API-First when:

You need strong collaboration, reuse, governance, and stable contracts for external consumers.



Key Takeaway

API-First puts the contract at the center of your API lifecycle, driving alignment, quality, and long-term success.



Collaborate



Design



Contract



Implement



Govern



Evolve

TRY IT YOURSELF — EXERCISE 5: SKETCH THE OPENAPI SKELETON

Reinforces: the OpenAPI document structure, paths, and schema slides you just walked, sketched for the file Lab 13 will author in full.

STEPS (notes/lab13-openapi-skeleton.md)

Step	What To Do
1 — Top-Level Keys	List the document sections: openapi, info, servers, paths, components.
2 — Two Paths	Sketch /customers and /customers/{id} with their operations.
3 — Customer Schema	List the Customer schema fields and their types, matching Amina's fixture.
4 — API-First	Say in one sentence why the contract is written before the controller.

OpenAPI skeleton

```
openapi: 3.0.3  info: {title, version}  servers: [...]
paths: /customers {get, post}  /customers/{id} {get, put, delete}
components.schemas: Customer{id,name,email,status}  ErrorResponse{...}
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-openapi-skeleton.md (workspace: examples/module-13-exercises).

REST API DESIGN BEST PRACTICES

Five practices that separate a maintainable enterprise API from a fragile one.

Practice	Why It Matters
Use nouns in URIs, verbs in HTTP methods	Keeps the interface predictable and RESTful
Return correct status codes	Lets clients branch on outcome without parsing the body
Version from day one	Avoids breaking every client on the first contract change
Paginate large collections	Protects the server and the client from unbounded payloads
Document with OpenAPI	Keeps the contract and the implementation from drifting apart

KEY TAKEAWAY None of these practices is optional in an enterprise API -- skipping one just moves the pain to production.

REST API Design Best Practices

Well-designed APIs are **consistent**, **predictable**, and **easy to use**.
Follow these practices to build robust and enterprise-ready REST APIs.

Key Best Practices



1. Resource-Oriented Design

Model APIs around resources, not actions.

Good:
GET /employees
Bad:
GET /getEmployees



2. Use Nouns, Not Verbs

Use nouns in URIs to represent resources.

Good:
/employees
Bad:
/getEmployees



3. Consistent URI Structure

Maintain a consistent and predictable URI pattern.

Good:
/departments/{id}/employees
Bad:
/departments/id/employees



4. Use Standard HTTP Methods

Use appropriate HTTP methods for each operation.

GET, POST, PUT, PATCH, DELETE



5. Use Proper Status Codes

Return meaningful HTTP status codes for all responses.

200 OK, 201 Created,
400 Bad Request,
404 Not Found, etc.



6. Support Filtering, Sorting and Pagination

Make APIs practical for real-world data usage.

/employees?dept=IT
&status=active
&sort=name
&page=1&size=20



7. Version Your APIs

Plan for change by using a clear versioning strategy.

/v1/employees
Accept: application/
vnd.company.v1+json



8. Design for Security

Secure your APIs with authentication, authorization, and validation.

OAuth2 / JWT
HTTPS
Input Validation



9. Consistent Error Responses

Return a consistent error structure to help clients handle failures.

```
{
  "error": "Not Found",
  "message": "Employee
with id 100 not found"
}
```



10. Clear and Up-to-Date Documentation

Keep API documentation accurate, examples rich, and easy to follow.

Use OpenAPI
(Swagger) to
document APIs

Additional Guidelines



Keep URIs short, readable, and intuitive.



Use lowercase letters in URIs.



Maintain backward compatibility whenever possible.



Use JSON as the default data format.



Optimize for performance (caching, compression, etc.).



Provide meaningful response messages and logs.



Key Takeaway

Great API design improves developer experience, ensures system scalability, and strengthens security and maintainability across the enterprise.



Consistent



Secure



Performant



Maintainable



Trusted

COMMON REST API DESIGN MISTAKES

Five recurring mistakes, and the concrete consequence each one causes for real clients.

Mistake	Consequence
Verbs in URIs (/getEmployee)	Confuses clients and breaks REST tooling expectations
Returning 200 for every outcome	Clients can't distinguish success from failure without parsing text
No pagination on large collections	Slow responses and out-of-memory clients as data grows
Inconsistent field naming	Every client integration needs special-case parsing
Breaking changes without a new version	Every existing consumer breaks at once

KEY TAKEAWAY Most REST design mistakes trace back to treating the URI or the status code as decoration instead of contract.

Common REST API Design Mistakes

Avoid these frequent pitfalls to build clean, consistent, and enterprise-ready APIs.



1. Poor URI Design

Using verbs or actions in URIs (e.g., `/getEmployees`, `/createUser`) instead of resource nouns.

✗ Bad: `/getEmployees` | ✓ Good: `/employees`



2. Not Resource-Oriented

Designing around operations instead of resources, leading to RPC-style endpoints.

✗ Bad: `/createEmployee` | ✓ Good: `/employees`



3. Inconsistent Naming

Using different naming styles, plural/singular inconsistently, or mixing conventions.

✗ Bad: `/User`, `/usersList`, `/userInfo` | ✓ Good: `/users`



4. Overusing Query Parameters

Passing too much data in query parameters instead of using the request body or proper filtering.

✗ Bad: `/search?name=John&age=30&...` | ✓ Good: `Filter`



5. Incorrect HTTP Methods

Using GET for create/update or POST for delete operations breaks REST principles and expectations.

✗ Bad: GET `/users` (create) | ✓ Good: POST `/users`



6. Leaking Implementation Details

Exposing internal IDs, table names, stack traces, or exception messages in API responses.

✗ Bad: SQL errors in response | ✓ Good: Clean error



7. Ignoring HTTP Status Codes

Returning 200 OK for every response or using wrong status codes reduces clarity for API consumers.

✗ Bad: 200 for errors | ✓ Good: Appropriate codes



8. Inconsistent Response Format

Returning different structures for similar responses makes client integration difficult.

✗ Bad: Varying response shapes | ✓ Good: Consistent JSON



9. Missing Pagination, Sorting or Filtering

Returning huge datasets without pagination or allowing sorting/filtering leads to performance issues.

✗ Bad: Large unfiltered lists | ✓ Good: Page, sort, filter



Pro Tip: Design for your consumers. Keep APIs simple, consistent, well-documented, and aligned with REST principles.



A good API is easy to use, hard to misuse, and easy to evolve.

ENTERPRISE REST API DESIGN CHECKLIST

Run this checklist before any new endpoint ships to the Employee Management API.

Checklist Item	Done When
Resources named with plural nouns	No verbs appear anywhere in a URI
Correct HTTP method per operation	GET/POST/PUT/PATCH/DELETE map exactly to CRUD intent
Status codes match outcome	2xx/4xx/5xx are never used interchangeably
Errors follow one JSON shape	Every failure returns the same error field set
Versioned from the first release	/api/v1/ prefix is in place before any client integrates
OpenAPI document kept current	Docs and implementation are reviewed together

KEY TAKEAWAY Run this checklist before any new endpoint ships -- catching drift here is cheaper than a breaking-change apology later.

Enterprise REST API Design Checklist

Use this checklist to ensure your REST APIs are consistent, secure, scalable, and aligned with enterprise standards.



1		Resource-Oriented Design • URIs represent nouns (resources), not actions. • Resources are identified clearly and uniquely.		9		API Documentation • Maintain up-to-date OpenAPI/Swagger documentation. • Document endpoints, models, parameters, and responses.	
2		Consistent URI Structure • Use plural nouns and consistent path patterns. • Avoid verbs, file extensions, and ambiguous URIs.		10		Security • Enforce authentication and authorization. • Use HTTPS and secure headers.	
3		HTTP Methods Used Correctly • Use appropriate HTTP methods for each operation. • Follow safe, idempotent, and cacheable principles.		11		Data Validation • Validate all input at the boundary. • Return clear validation errors to clients.	
4		Standardized Request & Response • Use consistent JSON structures across all APIs. • Follow proper naming conventions for fields.		12		Consistency • Follow naming, formatting, and response conventions. • Ensure behavior is predictable across all endpoints.	
5		Appropriate HTTP Status Codes • Return meaningful and accurate status codes. • Use 4xx for client errors and 5xx for server errors.		13		Performance & Scalability • Support caching where appropriate. • Design for efficient queries and minimal payloads.	
6		Error Handling • Return structured error responses with message, code, and details. • Do not expose internal stack traces or sensitive information.		14		Stateless & Layered Architecture • Keep APIs stateless. • Maintain separation of concerns across layers.	
7		Filtering, Sorting & Pagination • Support filtering, sorting, and pagination for collections. • Use standard query parameter conventions.		15		Monitoring & Observability • Log requests, errors, and performance metrics. • Correlate requests using trace or correlation IDs.	
8		Versioning Strategy • Follow a consistent API versioning approach. • Communicate version and deprecation policies.		16		Maintainability • Keep APIs simple, cohesive, and easy to evolve. • Follow enterprise coding and design standards.	



Goal: Deliver APIs that are easy to consume, secure by design, well-documented, and built for long-term enterprise success.



Consistent
Experience



Secure
by Design



Scalable
& Reliable

TRY IT YOURSELF — EXERCISE 6: LAB 13 READINESS SELF-CHECK

Capstone pre-lab gate: check your design against the best-practice and common-mistake slides, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab13-readiness.md)

Step	What To Do
1 — Mistake Sweep	Check your URI table against the common-mistakes slide and fix anything that matches.
2 — Deliverable Recall	List the Lab 13 deliverables from memory: OpenAPI file, URI table, status map, error contract.
3 — Fixture Check	Confirm your schema examples use Amina CUS-1001 and Ravi CUS-1002, never real data.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Readiness gate

```
uri-table  method-status-map  error-contract
collection-versioning  openapi-skeleton      -> all five complete?
examples use CUS-1001 / CUS-1002 only; no real customer data
```

TRY IT NOW — Complete Exercise 6 before Lab 13: exercises/exercise-06-lab13-readiness.md (workspace: examples/module-13-exercises).

LAB OVERVIEW — REST API DESIGN WITH JAVA

Design Customer REST resources + OpenAPI sketch + error/pagination contracts |

BUSINESS SCENARIO

- Northstar's Angular CRM must call a predictable Customer API. Your lead freezes:

LEARNING OBJECTIVES

- Identify Customer collection and item resources with consistent URIs Map create/read/update/delete (and activate) to correct HTTP methods and status codes Sketch an OpenAPI 3 document with paths, schemas, and examples for `CUS-1001` / `CUS-1002` Define a reusable error response that carries correlation id `lab-request-001` Design pagination, filtering, and sorting query parameters for list endpoints

WHAT YOU BUILD

- `examples/lab13-crm/` — OpenAPI, design docs, optional stub | OpenAPI for CUS-1001/1002 · design notes · optional thin Boot stub |

KEY TAKEAWAY This Module 13 lab freezes the ****Northstar Customer**** HTTP contract before heavy Spring coding: resources, URIs, methods, status codes, a shared error body, list query parameters, and an OpenAPI 3 document the Angular client and GitHub Actions checks can later...

Lab Overview – REST API Design

In this lab, you will design a clean, scalable, and enterprise-ready REST API for an **Employee Management System** by applying best practices and REST principles.



Lab Goal

Design a well-structured REST API for an Employee Management System with a focus on resources, URIs, HTTP methods, status codes, and consistent responses.



You Will Practice

- ✓ Identify and model API resources
- ✓ Design clean and consistent URI structure
- ✓ Map HTTP methods to operations
- ✓ Design request and response payloads (JSON)
- ✓ Define appropriate HTTP status codes
- ✓ Apply pagination, filtering, and sorting
- ✓ Plan versioning and error response strategy
- ✓ Document API using OpenAPI (Swagger)



Tech & Tools

- OpenAPI / Swagger Editor
- Postman or Insomnia
- JSON Formatter
- Design-first approach (no implementation required)



Scenario

You are designing a REST API for an Employee Management System that supports managing employees, departments, and job roles.



What You Will Deliver

- Resource model and URI design
- API endpoint specification
- Request and response examples
- Error response design
- OpenAPI (Swagger) documentation

API Scope



Core Resources

- Employees
- Departments
- Jobs / Roles
- (Optional) Employee Assignments



Example Endpoints (Sample)

GET /api/v1/employees
GET /api/v1/employees/{id}
POST /api/v1/employees
PUT /api/v1/employees/{id}
PATCH /api/v1/employees/{id}
DELETE /api/v1/employees/{id}



Cross-Cutting Concerns

- Pagination (page, size)
- Filtering (e.g., department, role)
- Sorting (e.g., name, createdAt)
- Consistent error responses
- Versioning strategy

Lab Approach



1. Analyze
business needs
and resources



2. Design
API contracts
and URIs



3. Define
methods and
responses



4. Document
using OpenAPI
(Swagger)



Outcome: A complete, well-documented REST API design ready for review and implementation.

LAB TASKS AND DELIVERABLES

REST API Design with Java — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy starter/ into java-bootcamp/examples/lab13-crm/.
3	Fill every // TODO / TODO: in OpenAPI + design notes.
4	Validate YAML (editor or npx @redocly/cli lint if available); evidence under notes/screenshots/lab-13/.
5	Mark timed-path Pass criteria in the starter README.
6	openapi/northstar-crm-customers.yaml (Customer collection + item)
7	Design notes: URI table, method/status map, error contract
8	Pagination / filter / sort query design (documented + reflected in OpenAPI)
9	Evidence that Amina/Ravi schemas match fixtures
10	Optional: thin Boot stub that serves one GET
11	README run/cleanup
12	No secrets or target/ committed

EXPECTED DELIVERABLES

- openapi/northstar-crm-customers.yaml (Customer collection + item) Design notes: URI table, method/status map, error contract
Pagination / filter / sort query design (documented + reflected in OpenAPI) Evidence that Amina/Ravi schemas match fixtures
Optional: thin Boot stub that serves one GET
README run/cleanup No secrets or target/ committed

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 13; complete pre-lab exercises 1→6 first, then the graded lab.

Lab Tasks and Deliverables

Complete the tasks below to design a well-structured, consistent, and enterprise-ready REST API.



LAB TASKS		
1		Analyze Requirements Review the Employee Management System requirements and identify resources and operations.
2		Identify Resources List all resources (e.g., employees, departments, jobs, roles) and define their relationships.
3		Design URI Structure Create clean, consistent, resource-based URIs following REST best practices.
4		Map HTTP Methods Map CRUD operations to appropriate HTTP methods for each resource.
5		Design Requests and Responses Define JSON request and response payloads, including success and error formats.
6		Define Status Codes Assign appropriate HTTP status codes for all success, client error, and server error scenarios.
7		Add Query Features Design pagination, filtering, and sorting parameters for collection endpoints.
8		Plan Versioning and Documentation Choose a versioning strategy and document the API using OpenAPI (Swagger).

DELIVERABLES		
1		Resource Model List of resources with attributes and relationships.
2		API Endpoint Specification Complete list of endpoints with URIs, HTTP methods, and descriptions.
3		Request & Response Examples Sample JSON payloads for success and error scenarios.
4		Status Code Matrix Matrix of all endpoints with possible HTTP status codes.
5		Query Parameter Specification Definition of pagination, filtering, and sorting parameters.
6		OpenAPI (Swagger) Document Complete OpenAPI 3.0+ specification of the designed API.

BONUS (OPTIONAL)

- ✓ Add example cURL or Postman requests for key endpoints.
- ✓ Include error response examples for validation failures.
- ✓ Add notes for authentication/authorization (if applicable).

```
curl -X GET https://api.example.com/v1/employees \
-H "Accept: application/json"
```

MODULE 13 SUMMARY

REST constraints, resource design, HTTP semantics, and OpenAPI, applied to one running API.

KEY CONCEPTS COVERED



REST Constraints

Client-server, stateless, cacheable, uniform interface, and layered system, all satisfied deliberately.



Resource-Based URIs

Plural nouns, consistent casing, and hierarchy that mirrors ownership -- never a verb in a path.



HTTP Methods

GET, POST, PUT, PATCH, and DELETE, each with distinct safety and idempotency guarantees.



Status Codes

2xx, 4xx, and 5xx used precisely, never defaulted to 200 for convenience.



Consistent JSON

One field-naming convention and one error-response shape across every endpoint.



OpenAPI Contract

Paths, operations, and reusable schemas documenting the API for every consumer.

MODULE FLOW RECAP

1

1. Constraints

2

2. Resources

3

3. HTTP Verbs

4

4. Status Codes

5

5. OpenAPI Docs

KEY TAKEAWAY Enterprise REST APIs succeed when resource design and HTTP semantics are applied consistently, not just once.

Module Summary

In this module, you explored how to design **clean, consistent, and enterprise-ready** REST APIs by applying REST architectural principles, HTTP semantics, and industry best practices.



What We Covered

- ✓ What REST is and why it matters
- ✓ REST architectural constraints and client-server principles
- ✓ Resource identification and URI design
- ✓ HTTP methods and their correct usage
- ✓ Request/response structure and JSON payloads
- ✓ HTTP status codes and error handling
- ✓ Pagination, filtering, sorting, and versioning
- ✓ OpenAPI (Swagger) and API-first design
- ✓ REST API best practices and common pitfalls



Key Takeaways

- ✓ Design around **resources**, not operations. Use nouns in URIs, not verbs.
- ✓ Use the **right HTTP methods** to reflect the intent of the operation.
- ✓ Maintain consistency in URIs, naming, responses, and error formats.
- ✓ Return **meaningful HTTP status codes** to improve client experience.
- ✓ Support **scalability** with **pagination**, filtering, sorting, and versioning.
- ✓ Document your API clearly with **OpenAPI** to enable discoverability and reuse.



Outcomes

-  You can design resource-based APIs that follow REST principles.
-  You can define clean, predictable, and consistent API contracts.
-  You can design robust error handling and status code strategies.
-  You can produce professional API documentation using OpenAPI.
-  You are ready to implement and evolve APIs in enterprise environments.

How It All Fits Together



Understand Users

Know who your consumers are and their needs.



Model Resources

Identify and structure your API resources.



Apply HTTP Semantics

Use methods, headers, and status codes correctly.



Design & Document

Define contracts and document using OpenAPI.



Deliver Value

Build scalable, reliable, and consumer-friendly APIs.



Remember: A well-designed API is easy to use, hard to misuse, and easy to evolve.



Great APIs are built with clarity, consistency, and care.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of REST constraints, resource design, and URI conventions.

1. Which of these is NOT one of Fielding's mandatory REST constraints?

- A. Stateless communication
- B. Layered system
- C. Code on demand**
- D. Uniform interface

3. What does the stateless constraint actually require?

- A. The server may never use a database
- B. Every request carries all context the server needs**
- C. Clients may never send an Authorization header
- D. Responses may never be cached

2. Which URI follows resource-based REST design correctly?

- A. /getEmployeeById?id=42
- B. /employees/42**
- C. /employee/fetch/42
- D. /EmployeeData?empld=42

4. In /departments/{id}/employees, what does the URI hierarchy express?

- A. An unrelated pair of resources
- B. That employees belong to a department**
- C. A required query parameter
- D. A deprecated endpoint

KEY TAKEAWAY Code on demand is optional, not mandatory; correct URIs use nouns and path variables, never verbs or query-string ids.

KNOWLEDGE CHECK (2 OF 2)

Test your understanding of HTTP methods, status codes, and OpenAPI documentation.

1. Which HTTP method is idempotent but NOT safe?

- A. GET
- B. PUT**
- C. POST
- D. PATCH (with an increment)

3. What is the correct response when a client sends valid credentials but lacks permission for the action?

- A. 401 Unauthorized
- B. 403 Forbidden**
- C. 404 Not Found
- D. 400 Bad Request

2. A POST /employees call succeeds and creates a new record. What status code, and what header, should the response include?

- A. 200, with no extra header
- B. 201, with a Location header**
- C. 204, with no body
- D. 202, with a Retry-After header

4. In an OpenAPI document, where would the reusable Employee JSON schema be defined?

- A. Inside every path that uses it
- B. Under components/schemas**
- C. In the servers section
- D. In the info section

KEY TAKEAWAY 201+Location on create, 403 for 'authenticated but not allowed', and one shared schema under components -- these are the details that separate a correct API from a merely working one.

Knowledge Check

Test your understanding of REST API design concepts covered in this module.



1 What is the primary principle of REST architectural style?

- ☐ A Maintain state on the server
- ☐ B Use SOAP for communication
- ☐ C Stateless communication
- ☐ D Tight coupling between client and server

2 Which HTTP method should be used to create a new resource?

- ☐ A GET
- ☐ B PUT
- ☐ C POST
- ☐ D DELETE

3 What status code indicates that a request was successful and a resource was created?

- ☐ A 200 OK
- ☐ B 201 Created
- ☐ C 204 No Content
- ☐ D 400 Bad Request

4 Which of the following is an example of a good resource URI?

- ☐ A /getEmployees
- ☐ B /employees/listAll
- ☐ C /v1/departments/123/employees
- ☐ D /createEmployee

5 Which HTTP method is idempotent by nature?

- ☐ A POST
- ☐ B PATCH
- ☐ C PUT
- ☐ D DELETE

6 What is the recommended format for API response payloads?

- ☐ A XML
- ☐ B HTML
- ☐ C JSON
- ☐ D Plain Text

7 Which header is used by the client to specify the expected response format?

- ☐ A Content-Type
- ☐ B Accept
- ☐ C Authorization
- ☐ D Cache-Control

8 Which approach is preferred for API versioning?

- ☐ A Add version in request body
- ☐ B Version in custom header
- ☐ C Version in URI path
- ☐ D Use query parameter only



Think & Reflect

- Why is statelessness important in REST APIs?
- How does using proper HTTP status codes improve API usability?
- What are the risks of inconsistent API design?



Challenge

Design a GET endpoint to fetch all employees in a department with pagination, sorting by name, and filtering by role. Write the URI and example query parameters.

Answer Key

1. C 2. C 3. B 4. C 5. C 6. C 7. B 8. C



Great APIs are designed with clarity, consistency, and the consumer in mind.

*"A well-designed API is a promise you keep
for years, not a guess you make once."*

MODULE 13 COMPLETE

REST API Design with Java

You can now design resource-based REST APIs with correct HTTP semantics, status codes, and OpenAPI documentation.